The two-machine flow shop with maximum weighted completion time: structure, algorithms and a computational study

Author One[a,*], Author Two[b]

[a]Institution One, Location, Country

[b]Institution Two, Location, Country

Abstract

The two-machine flow shop with maximum weighted completion time ($F2 \parallel WC_{\max}$) models a line dominated by a few high-priority orders. We give a mixed-integer linear programming (MILP) formulation and prove that the problem is strongly NP-hard. An optimal schedule can be taken to be a permutation schedule with one block per weight, Johnson's order within each, and non-increasing weight order across blocks. An exact dynamic program runs in $O(nK2^K V^{3K-3})$ time, where K is the number of distinct weights and V the total processing time; we add a 2-approximation in $O(n \log n)$ time and an FPTAS for every fixed K, and for large instances we adapt NEH and develop a genetic algorithm. In 440 small and 640 large instances, the MILP model settles 436 of the small ones to optimality; on a separate $K = 2$ set, the dynamic program certifies optimality up to 80 jobs for processing times in $[1, 40]$ and $[1, 80]$, and up to 100 for $[1, 40]$, the limit being time and memory rather than correctness. On small instances the 2-approximation averages 5.8 per cent above the reference and NEH 3.1 per cent; on large ones, against a lower bound, they average 2.7 and 5.1 per cent. The genetic algorithm has the largest gap at small sizes but becomes the closest method at the top of that scale, and it overtakes both rules at roughly 220 to 240 jobs, at a much higher cost.

Keywords: Scheduling; Two-machine flow shop; Dynamic programming; Weighted makespan;

*Corresponding author. E-mail address: author@email.com

1 Introduction

Scheduling plays a critical role in semiconductor wafer fabrication, where each wafer lot undergoes hundreds of processing operations and its completion time determines the delivery time of the corresponding customer order. In practice, high-priority hot lots must be completed ahead of regular lots, and the penalties associated with delays may vary across customers and processing layers. Scheduling decisions should therefore not only improve overall production performance but also prevent excessive delays to high-priority lots. Two commonly studied objectives in this context are the total weighted completion time, which minimizes the sum of weighted completion times, and the maximum weighted completion time, which minimizes the largest weighted completion time. Unlike the conventional makespan, the maximum weighted completion time accounts for differences in lot priorities and focuses on the worst-case weighted completion performance. When all lots have unit weights, it reduces to the conventional makespan.

Research on the maximum weighted completion time has primarily focused on single-machine and parallel-machine environments. For single-machine scheduling, approximation algorithms and polynomial-time approximation schemes have been developed in two-agent settings (Nong et al., 2011), while online variants have been extensively investigated, with competitive algorithms established for both preemptive and non-preemptive settings (Li, 2015). Online algorithms with best-possible guarantees were subsequently developed (Chai et al., 2018). Later studies have considered release dates, preemption, and multiple objectives in multi-agent scheduling (Yuan et al., 2020), lot scheduling with the maximum weighted tardiness objective (Mosheiov and Sarig, 2023), and identical parallel machines (Sun, 2024). More recent work has addressed order acceptance and rejection decisions (Lu et al., 2025), lot scheduling under the same objective (Zheng et al., 2025), online scheduling on an unbounded parallel-batch machine (Zhang et al., 2025), online settings under (dis)agreeability conditions (Fan et al., 2026), and scheduling with restricted rate- or weight-modifying activities (Zuo et al., 2026a,b). Despite these advances, the maximum weighted completion time objective has received limited attention in flow-shop environments, particularly in the two-machine setting. This gap is practically relevant to semiconductor manufacturing, where delays in high-priority lots may postpone the delivery of entire customer orders. Motivated by this application, we investigate the two-machine flow-shop scheduling problem with the objective of minimizing the maximum weighted completion time.

The two-machine flow shop has been extensively studied since Johnson established his rule for makespan minimization (Johnson, 1954). Subsequent research has developed constructive heuristics for various flow-shop settings (Shao et al., 2021), while the corresponding three-machine problem remains strongly $\mathcal{NP}$-hard (Pinedo, 2022). Genetic algorithms have likewise been adapted to related flow-shop settings (Qiao et al., 2022; Wang et al., 2023). Research has also extended the problem to settings involving disruptions (Hnaien

and Arbaoui, 2023), branch-and-bound methods (Liu and Urgo, 2024a), reentrant production (Liu and Urgo, 2024b), alternative objective functions (Shabtay and Gerstl, 2024), refined NEH constructions (Puka et al., 2024), and distributed manufacturing environments (Becker et al., 2025). Meanwhile, refinements of Johnson's rule continue to attract research interest (Della Croce and Schau, 2026). These production structures arise in various industries, including semiconductor manufacturing, where lots undergo lithography followed by etching, and textile manufacturing, where dyeing precedes drying.

This paper makes five main contributions. First, we formulate the problem as a mixed-integer programming model. Second, we establish its strong $\mathcal{NP}$-hardness. Third, we derive a structural theorem characterizing an optimal schedule. Fourth, based on this structure, we develop three algorithms: an exact dynamic programming algorithm, a 2-approximation algorithm, and an approximation scheme. Finally, we propose an adapted NEH heuristic and a genetic algorithm, and conduct a computational study involving 440 small-scale and 640 large-scale instances. The remainder of the paper is organized as follows. Section 2 presents the problem formulation, and Section 3 establishes its computational complexity. Section 4 presents the structural theorem and the associated algorithms, while Section 5 describes the heuristic methods. Section 6 reports the computational results, and Section 7 concludes the paper.

2 Problem description

Consider a set $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ of n independent and non-preemptive jobs to be scheduled in a two-machine flow shop with machines M_1 and M_2. All jobs are released at time zero and follow the same route, $M_1 \to M_2$. For $j = 1, 2, \ldots, n$, job J_j is represented by a triple $\Phi_j = (a_j, b_j, w_j)$, where a_j and b_j are its nonnegative integer processing times on M_1 and M_2, respectively, and w_j is a weight reflecting its importance. Let K be the number of distinct weights. For a feasible schedule π, let $C_j^1(\pi)$ and $C_j^2(\pi)$ denote the completion times of J_j on M_1 and M_2, respectively; thus $C_j(\pi) = C_j^2(\pi)$ is its completion time. The makespan and the weighted makespan (i.e., the maximum weighted completion time) in π are given by

$$C_{\max}(\pi) = \max_{1 \le j \le n} \{C_j(\pi)\} \quad \text{and} \quad WC_{\max}(\pi) = \max_{1 \le j \le n} \{w_j C_j(\pi)\},$$

respectively. The objective is to minimize $WC_{\max}$. Using the standard 3-field scheduling notation (Pinedo, 2022), this problem is denoted by $F2 \parallel WC_{\max}$.

In a two-machine flow shop, a schedule is called a *permutation* schedule if the jobs are processed in the same order on both machines. Johnson (1954) showed that an optimal permutation schedule exists for $F2 \parallel C_{\max}$. By a similar argument, an optimal permutation schedule also exists for $F2 \parallel WC_{\max}$. Hence, we restrict attention to permutation schedules.

Let $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ denote a feasible permutation schedule for $F2 \parallel WC_{\max}$, where $J_{\pi(j)}$ is the j-th job in π. By simple calculation, it follows that

$$C_{\pi(j)} = \max_{1 \le i \le j} \Big\{ \sum_{h=1}^{i} a_{\pi(h)} + \sum_{h=i}^{j} b_{\pi(h)} \Big\}, \quad j = 1, 2, \ldots, n. \tag{1}$$

Then, the weighted makespan of π is given by

$$WC_{\max}(\pi) = \max_{1 \le j \le n} \{ w_{\pi(j)} C_{\pi(j)} \}. \tag{2}$$

To formulate the problem as a mixed integer linear program (MILP), we introduce the following decision variables:

- $x_{jk} = 1$ if job J_j is the k-th job, and 0 otherwise;
- $C^1_{[k]}$ is the completion time of the k-th job on M_1;
- $C^2_{[k]}$ is the completion time of the k-th job on M_2;
- C_j is the completion time of job J_j;
- M is a sufficiently large constant (e.g., $M = \sum_{j=1}^{n} (a_j + b_j)$).

The MILP model can be formulated as follows:

$$\min \quad WC_{\max} \tag{3}$$

$$\text{s.t.} \quad \sum_{k=1}^{n} x_{jk} = 1, \quad j = 1, 2, \ldots, n \tag{4}$$

$$\sum_{j=1}^{n} x_{jk} = 1, \quad k = 1, 2, \ldots, n \tag{5}$$

$$C^1_{[1]} = \sum_{j=1}^{n} a_j x_{j1} \tag{6}$$

$$C^1_{[k]} = C^1_{[k-1]} + \sum_{j=1}^{n} a_j x_{jk}, \quad k = 2, 3, \ldots, n \tag{7}$$

$$C^2_{[1]} = C^1_{[1]} + \sum_{j=1}^{n} b_j x_{j1} \tag{8}$$

$$C^2_{[k]} \ge C^1_{[k]} + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, 3, \ldots, n \tag{9}$$

$$C^2_{[k]} \ge C^2_{[k-1]} + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, 3, \ldots, n \tag{10}$$

$$C_j \ge C^2_{[k]} - M(1 - x_{jk}), \quad j = 1, 2, \ldots, n; \quad k = 1, 2, \ldots, n \tag{11}$$

$$WC_{\max} \geq w_j C_j, \quad j = 1, 2, \ldots, n \tag{12}$$

$$x_{jk} \in \{0, 1\}, \quad j = 1, 2, \ldots, n; \quad k = 1, 2, \ldots, n \tag{13}$$

$$C_j, C^1_{[k]}, C^2_{[k]} \geq 0 \quad j = 1, 2, \ldots, n; \quad k = 1, 2, \ldots, n \tag{14}$$

The objective is to minimize $WC_{\max}$, as expressed in (3). Constraints (4) and (5) assign each job to exactly one position and each position to exactly one job. Constraints (6) and (7) determine the completion times on M_1, and (8) through (10) determine those those on M_2. In particular, for $k \geq 2$, the job in position k can start on M_2 only after it has completed M_1 and the preceding job has finished on M_2. Constraint (11) relates the position-based completion times to the job-based ones via a big-M formulation. Note that when $x_{jk} = 1$ (i.e., J_j is assigned to the k-th position), it requires $C_j \geq C^2_{[k]}$; when $x_{jk} = 0$, it is inactive, since $C^2_{[k]} - M \leq 0$ and $C_j \geq 0$. Constraint (12) imposes $WC_{\max} \geq w_j C_j$ for each job, so minimizing $WC_{\max}$ yields the optimal objective value of (2). Constraints (13) and (14) require the decision variables x_{jk} to be binary and the completion times nonnegative.

3 Computational complexity

When all jobs have unit weights (i.e., $w_j = 1$ for $j = 1, 2, \ldots, n$), $F2 \parallel WC_{\max}$ reduces to $F2 \parallel C_{\max}$, which is solvable in $O(n \log n)$ time (Johnson, 1954). In contrast, We show that $F2 \parallel WC_{\max}$ is ordinarily $\mathcal{NP}$-hard even with only two distinct weights and strongly $\mathcal{NP}$-hard when K is arbitrary.

Theorem 3.1. *$F2 \parallel WC_{\max}$ is ordinarily $\mathcal{NP}$-hard even with only two distinct weights.*

Proof. To establish the result, we give a polynomial-time reduction from PARTITION, which is $\mathcal{NP}$-complete in the ordinary sense (Garey and Johnson, 1979).

PARTITION: Given r positive integers $x_1, x_2, \ldots, x_r$ and an integer X with $\sum_{i=1}^{r} x_i = 2X$, does there exist a partition of $\{1, 2, \ldots, r\}$ into two disjoint subsets $\mathcal{X}_1$ and $\mathcal{X}_2$ such that $\sum_{i \in \mathcal{X}_1} x_i = \sum_{i \in \mathcal{X}_2} x_i = X$?

Given an arbitrary instance $(x_1, x_2, \ldots, x_r; X)$ of PARTITION, we construct an instance of $F2 \parallel WC_{\max}$ with $n = r + 1$ jobs. For $j = 1, 2, \ldots, n-1$, job J_j has $a_j = 1$, $b_j = r x_j$, and $w_j = X + 1$; job J_n has $a_n = rX$, $b_n = 1$, and $w_n = 2X + 1$. Set the threshold to $Y = r(X + 1)(2X + 1)$. The question is whether there exists a schedule with $WC_{\max} \leq Y$.

We demonstrate that $F2 \parallel WC_{\max}$ has a feasible schedule π with $WC_{\max}(\pi) \leq Y$ if and only if PARTITION has a solution.

Suppose that PARTITION admits a solution $(\mathcal{X}_1, \mathcal{X}_2)$ with $\sum_{j \in \mathcal{X}_1} x_j = \sum_{j \in \mathcal{X}_2} x_j = X$. Define $\mathcal{J}_1 = \{J_j : j \in \mathcal{X}_1\}$ and $\mathcal{J}_2 = \{J_{r+j} : j \in \mathcal{X}_2\}$. We build a feasible permutation schedule π in which both machines process the jobs in the order $\mathcal{J}_1 \to J_n \to \mathcal{J}_2$, with the jobs in $\mathcal{J}_1$ and $\mathcal{J}_2$ processed in arbitrary order (see Figure 1).

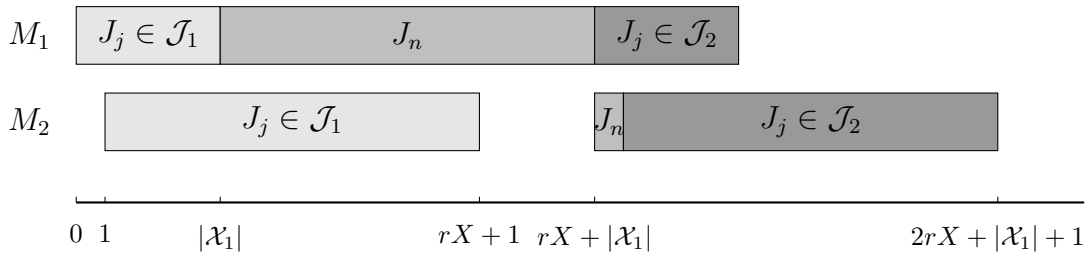


Figure 1: A feasible permutation scheule π

Since $a_j \le b_j$ for each $j = 1, 2, \ldots, r$ and $|\mathcal{X}_1| \le r-1$, it can be easily verified that $C_n(\pi) = rX + |\mathcal{X}_1| + 1 \le r(X+1)$ and $C_{\max}(\pi) = 2rX + |\mathcal{X}_1| + 1 \le r(2X+1)$. Hence, $WC_{\max}(\pi) \le r(X+1)(2X+1) = Y$.

Conversely, suppose that $F2 \parallel WC_{\max}$ admits a feasible schedule π with $WC_{\max}(\pi) \le Y$. Let $\mathcal{J}_1 = \{J_j : C_j(\pi) < C_n(\pi)\}$ and $\mathcal{J}_2 = \{J_j : C_j(\pi) > C_n(\pi)\}$ be the sets of jobs completed before and after J_n in π, respectively, and let $\mathcal{X}_1 = \{j : J_j \in \mathcal{J}_1\}$ and $\mathcal{X}_2 = \{j : J_j \in \mathcal{J}_2\}$. By (1), we have

$$C_n(\pi) \ge \sum_{j \in \mathcal{X}_1} a_j + a_n + 1 = |\mathcal{X}_1| + rX + 1 \tag{15}$$

and

$$C_n(\pi) \ge a_{\pi(1)} + \sum_{j \in \mathcal{X}_1} b_j + b_n = r \sum_{j \in \mathcal{X}_1} x_j + 2. \tag{16}$$

Case 1. If $\sum_{j \in \mathcal{X}_1} x_j > X$, then $\sum_{j \in \mathcal{X}_1} x_j \ge X + 1$ because the x_j are integers. By (16), $WC_{\max}(\pi) \ge w_n C_n(\pi) \ge (2X+1)(r(X+1)+2) > r(X+1)(2X+1) = Y$, contradicting the assumption that $WC_{\max}(\pi) \le Y$.

Case 2. If $\sum_{j \in \mathcal{X}_1} x_j < X$, then $\sum_{j \in \mathcal{X}_1} x_j \le X - 1$ because the x_j are integers. Since $(\mathcal{X}_1, \mathcal{X}_2)$ is a partition of $\{1, 2, \ldots, r\}$, we have

$$\sum_{j \in \mathcal{X}_2} x_j = 2X - \sum_{j \in \mathcal{X}_1} x_j \ge X + 1. \tag{17}$$

By (1), (15), and (17), we have

$$C_{\pi(n)}(\pi) = C_{\max}(\pi) \ge C_n(\pi) + \sum_{j \in \mathcal{X}_2} b_j \ge |\mathcal{X}_1| + rX + 1 + r \sum_{j \in \mathcal{X}_2} x_j \ge r(2X+1) + 1. \tag{18}$$

Using $w_{\pi(n)} = 1$, (18) yields

$$WC_{\max}(\pi) \ge w_{\pi(n)} C_{\pi(n)}(\pi) \ge (X+1)(r(2X+1)+1) > r(X+1)(2X+1) = Y,$$

This contradicts the assumption that $WC_{\max}(\pi) \le Y$.

Combining the above two cases, we obtain that $\sum_{j \in \mathcal{X}_1} x_j = \sum_{j \in \mathcal{X}_2} x_j = X$. Therefore, $(\mathcal{X}_1, \mathcal{X}_2)$ is a solution to PARTITION. □

Theorem 3.2. *$F2 \parallel WC_{\max}$ is strongly $\mathcal{NP}$-hard when K is arbitrary.*

Proof. To establish the result, we give a polynomial-time reduction from 3-PARTITION, which is strongly $\mathcal{NP}$-complete (Garey and Johnson, 1979).

3-PARTITION: Given $3r$ positive integers $x_1, x_2, \ldots, x_{3r}$ and an integer X with $\sum_{i=1}^{3r} x_i = rX$ and $\frac{X}{4} < x_j < \frac{X}{2}$ for each $j = 1, 2, \ldots, 3r$, does there exist a partition of $\{1, 2, \ldots, 3r\}$ into m disjoint subsets $\mathcal{X}_1, \mathcal{X}_2, \ldots, \mathcal{X}_r$ such that $\sum_{j \in \mathcal{X}_i} x_j = X$ for $i = 1, 2, \ldots, r$?

Given an arbitrary instance $(x_1, x_2, \ldots, x_{3r}; X)$ of 3-PARTITION, we construct an instance of $F2 \parallel WC_{\max}$ consisting of r separating jobs and $3r$ basic jobs. For $j = 1, 2, \ldots, r$, separating job J_j has $a_j = 4X$, $b_j = X$, and $w_j = \frac{5r+4}{5j}$; for $j = r+1, r+2, \ldots, 4r$, basic job J_j has $a_j = x_{j-r}$, $b_j = 4x_{j-r}$, and $w_j = 1$. Set the threshold to $Y = (5r+4)X$. The question is whether there exists a schedule with $WC_{\max} \leq Y$.

We demonstrate that $F2 \parallel WC_{\max}$ has a feasible schedule π with $WC_{\max}(\pi) \leq Y$ if and only if 3-PARTITION has a solution.

Suppose that 3-PARTITION admits a solution $(\mathcal{X}_1, \mathcal{X}_2, \ldots, \mathcal{X}_r)$, so that $\sum_{j \in \mathcal{X}_i} x_j = X$ and $|\mathcal{X}_i| = 3$ for all i. Let $\mathcal{J}_i = \{J_{r+j} : j \in \mathcal{X}_i\}$ for $i = 1, 2, \ldots, r$. We build a feasible permutation schedule π in which both machines process the jobs in the order

$$J_1 \to \mathcal{J}_1 \to J_2 \to \mathcal{J}_2 \to \cdots \to J_r \to \mathcal{J}_r,$$

with the jobs in each $\mathcal{J}_i$ processed in arbitrary order (see Figure 2).

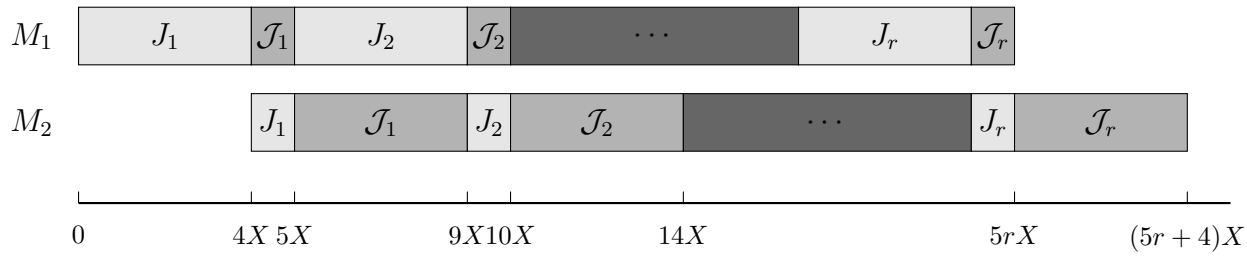


Figure 2: A feasible permutation scheule π

According to Figure 2, it can be easily verified that $C_j(\pi) = 5jX$ for $j = 1, 2, \ldots, r$, and $C_j(\pi) \leq C_{\max}(\pi) = (5r+4)X$ for $j = r+1, r+2, \ldots, 4r$. Then $w_j C_j(\pi) = (5r+4)X = Y$ for $j = 1, 2, \ldots, r$, and $w_j C_j(\pi) \leq w_j C_{\max}(\pi) = (5r+4)X = Y$ for $j = r+1, r+2, \ldots, 4r$. Hence, $WC_{\max}(\pi) \leq Y$.

Conversely, suppose that $F2 \parallel WC_{\max}$ admits a feasible schedule π with $WC_{\max}(\pi) \leq Y$. Because all separating jobs have the same processing time on M_1 and M_2 and weights $w_1 > w_2 > \cdots > w_r$, a standard interchange argument lets us assume they appear in π in decreasing order of weight.

Claim 1. $C_{\max}(\pi) \leq Y = (5r+4)X$.

Proof of Claim 1. Since $w_j \geq 1$ for all $1 \leq j \leq 4r$, we have $C_{\max}(\pi) \leq WC_{\max}(\pi) \leq Y = (5r+4)X$. ■

Claim 2. J_1 is the first job scheduled in π.

Proof of Claim 2. Since $WC_{\max}(\pi) \leq Y = (5r+4)X$ and $w_1 = \frac{5r+4}{5}$, we have $a_1 + b_1 \leq C_1(\pi) \leq \frac{WC_{\max}(\pi)}{w_1} \leq 5X = a_1 + b_1$. This means that J_1 is the first job scheduled in π. ■

For $i = 1, 2, \ldots, r-1$, let $\mathcal{B}_i = \{J_j : C_i(\pi) < C_j(\pi) < C_{i+1}(\pi)\}$ be the set of basic jobs scheduled between J_i and J_{i+1} in π, and let $\mathcal{B}_r = \{J_j : C_j(\pi) > C_r(\pi)\}$ be the set of basic jobs scheduled after J_r. For $i = 1, 2, \ldots, r$, define $\mathcal{X}_i = \{j - r : J_j \in \mathcal{B}_i\}$. Then, by Claim 2, π can be written as

$$\pi = (J_1 \to \mathcal{B}_1 \to J_2 \to \mathcal{B}_2 \to \cdots \to J_r \to \mathcal{B}_r). \tag{19}$$

Since $(\mathcal{X}_1, \mathcal{X}_2, \ldots, \mathcal{X}_r)$ is a partition of $\{1, 2, \ldots, 3r\}$, we have

$$\sum_{i=1}^{r} \sum_{j \in \mathcal{X}_i} x_j = \sum_{j=1}^{3r} x_j = rX. \tag{20}$$

Claim 3. For $k = 1, 2, \ldots, r-1$, $\sum_{i=1}^{k} \sum_{j \in \mathcal{X}_i} x_j \leq kX$.

Proof of Claim 3. Suppose to the contrary that there exists some m $(1 \leq m \leq r-1)$ with $\sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j > mX$. Since the x_j are integers, this implies

$$\sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j \geq mX + 1. \tag{21}$$

Because $a_j = 4X$ and $b_j = X$ for $1 \leq j \leq r$, and $a_j = x_{j-r}$ for $r+1 \leq j \leq 4r$, (1), (19), and (21) give

$$C_{m+1}(\pi) \geq \sum_{j=1}^{m+1} a_j + \sum_{i=1}^{m} \sum_{j \in \mathcal{B}_i} a_j + b_{m+1} = 4(m+1)X + \sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j + X \geq 5(m+1)X + 1. \tag{22}$$

Using $w_{m+1} = \frac{5r+4}{5(m+1)}$, (22) yields

$$WC_{\max}(\pi) \geq w_{m+1} C_{m+1}(\pi) \geq \frac{5r+4}{5(m+1)}(5(m+1)X + 1) > (5r+4)X = Y, \tag{23}$$

which contradicts the assumption that $WC_{\max}(\pi) \leq Y$. Hence, Claim 3 holds. ■

Claim 4. For $k = 1, 2, \ldots, r-1$, $\sum_{i=1}^{k} \sum_{j \in \mathcal{X}_i} x_j \geq kX$.

Proof of Claim 4. Suppose to the contrary that there exists some m $(1 \leq m \leq r-1)$ with $\sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j < mX$. By (20) and the x_j are integers, we have

$$\sum_{i=m+1}^{r} \sum_{j \in \mathcal{X}_i} x_j = rX - \sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j \geq (r-m)X + 1. \tag{24}$$

Because $a_j = 4X$ and $b_j = X$ for $1 \le j \le r$, and $a_j = x_{j-r}$ and $b_j = 4x_{j-r}$ for $r+1 \le j \le 4r$, (1), (19), (20), and (24) give

$$\begin{aligned} C_{\max}(\pi) &\ge \sum_{j=1}^{m+1} a_j + \sum_{i=1}^{m} \sum_{j \in \mathcal{B}_i} a_j + \sum_{j=m+1}^{r} b_j + \sum_{i=m+1}^{r} \sum_{j \in \mathcal{B}_i} b_j \\ &= 4(m+1)X + \sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j + (r-m)X + \sum_{i=m+1}^{r} \sum_{j \in \mathcal{X}_i} 4x_j \\ &= 4(m+1)X + (r-m)X + rX + 3 \sum_{i=m+1}^{r} \sum_{j \in \mathcal{X}_i} x_j \\ &\ge 4(m+1)X + (r-m)X + rX + 3((r-m)X + 1) \\ &= (5r+4)X + 3 > (5r+4)X = Y. \end{aligned}$$

which contradicts Claim 1. Hence, Claim 4 holds. ■

Combining Claims 3 and 4 with (20), we obtain that $\sum_{j \in \mathcal{X}_i} x_j = X$ for $i = 1, 2, \ldots, r$. Since $X/4 < x_j < X/2$ for each j, a sum equal to X must contain exactly three elements; hence $|\mathcal{X}_i| = 3$ for each i. Therefore, $(\mathcal{X}_1, \mathcal{X}_2, \ldots, \mathcal{X}_r)$ is a solution to 3-Partition. □

4 Exact and approximation algorithms

This section develops the exact dynamic program and the two approximation algorithms that rest on it. Throughout, K denotes the number of distinct weights and $V = \sum_{j=1}^{n} (a_j + b_j)$ the total processing time.

Since the objective is a maximum over weighted completion times, jobs of equal weight are interchangeable. An optimal schedule can therefore be assumed to have a block structure: one block per distinct weight, Johnson's order within each block, and blocks in non-increasing weight order. A schedule is fixed once each weight class has been assigned to a block, and the running time of every algorithm below is governed by the number K of those classes.

Three algorithms are built on this structure. The first is exact and prunes dominated states: a state records each block's processing time on both machines, the completion time of its last job on M_2, and that job's weight, and the dynamic program runs in $O(n \cdot K \cdot |\mathscr{H}|_{\max})$ time with $|\mathscr{H}|_{\max} \le 2^K V^{3K-3}$. The second sorts the jobs by non-increasing weight in $O(n \log n)$ time and returns a schedule within a factor 2 of the optimum. The third rounds the time coordinates and the weights to geometrically spaced grids, which turns the dynamic program into a $(1+\epsilon)$-approximation scheme for a fixed number of distinct weights.

4.1 Block structure of an optimal schedule

Within the permutation schedules of Section 2, Johnson's rule (Pinedo, 2022) minimizes the makespan: the jobs with $a_j \le b_j$ are scheduled first in non-decreasing order of a_j, and the remaining jobs follow in non-increasing order of b_j.

Fix a permutation schedule π. Let $W_1 > W_2 > \cdots > W_K$ be the K distinct weights occurring among the jobs, and let

$$\mathcal{G}_k = \{J_j \in \mathcal{J} \mid w_j = W_k\}, \qquad k = 1, \ldots, K,$$

denote the corresponding weight classes. Set $\ell_0 = 0$. For $k = 1, \ldots, K$ in sequence, let ℓ_k be the position of the last job of weight W_k among the jobs not yet assigned to a block, that is, among $\pi(\ell_{k-1}+1), \ldots, \pi(n)$; if no such job remains, set $\ell_k = \ell_{k-1}$. The blocks are the consecutive segments

$$\mathcal{B}_k = \{\pi(\ell_{k-1}+1), \ldots, \pi(\ell_k)\}, \qquad k = 1, \ldots, K,$$

so that in particular $\mathcal{B}_1 = \{\pi(1), \ldots, \pi(\ell_1)\}$. Equivalently, once the blocks $\mathcal{B}_1, \ldots, \mathcal{B}_k$ have been removed, the remaining jobs are

$$\mathcal{J} \setminus \bigcup_{i=1}^{k} \mathcal{B}_i = \{\pi(\ell_k+1), \ldots, \pi(n)\}, \qquad k = 1, \ldots, K.$$

Hence $\ell_1 \le \ell_2 \le \cdots \le \ell_K$, and no block extends beyond its own last job. For each block $\mathcal{B}_i$, let A_i and B_i denote the total processing time of its jobs on M_1 and M_2, respectively, C_i the completion time of its last job on M_2, and L_i the weight of its last job. If $\ell_k = \ell_{k-1}$, then $\mathcal{B}_k$ contains no job; such a block is called empty and satisfies $A_k = B_k = C_k = L_k = 0$, and it may host any job of weight at most W_k. By construction, a non-empty block $\mathcal{B}_k$ ends with a job of weight W_k, whereas an empty block has $L_k = 0$. Since $W_1 > W_2 > \cdots > W_K$, the block-end weights are non-increasing on the non-empty blocks.

We illustrate the block construction with a small example.

Example 4.1. *Consider a six-job instance with sequence $\pi = (J_1, \ldots, J_6)$ and the following (a_j, b_j, w_j) values:*

	J_1	J_2	J_3	J_4	J_5	J_6
a_j	2	5	6	8	4	3
b_j	5	7	9	4	3	2
w_j	2	2	5	2	4	2

The weight classes are $\mathcal{G}_1 = \{J_3\}$, $\mathcal{G}_2 = \{J_5\}$, and $\mathcal{G}_3 = \{J_1, J_2, J_4, J_6\}$, and the resulting blocks are $\mathcal{B}_1 = \{J_1, J_2, J_3\}$, $\mathcal{B}_2 = \{J_4, J_5\}$, and $\mathcal{B}_3 = \{J_6\}$.

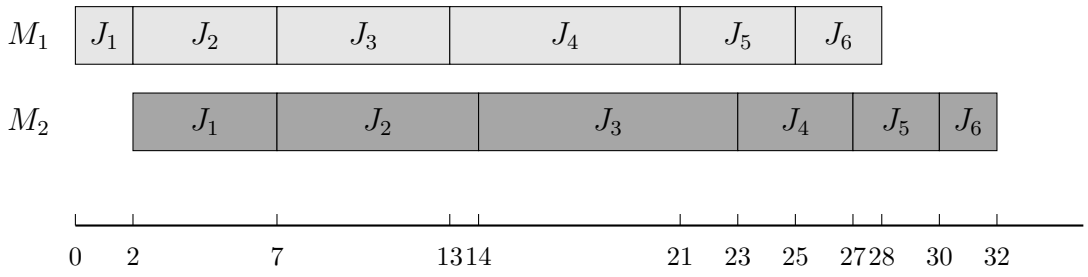


Figure 3

Lemma 4.1. *For any schedule π, scheduling the jobs of each block by Johnson's rule and concatenating the blocks in their original order gives a schedule π' with $WC_{\max}(\pi') \leq WC_{\max}(\pi)$.*

Proof. By construction, a non-empty block $\mathcal{B}_k$ ends with a job of weight L_k, and every job in $\mathcal{B}_k$ has weight at most L_k: a weight occurring in $\mathcal{B}_k$ either equals W_k, or it has already appeared in an earlier block, whose weights are larger. Let $C_{\pi(\ell_k)}$ denote the completion time of the last job of $\mathcal{B}_k$ in π, the job that sits at position ℓ_k. An empty block satisfies $L_k C_{\pi(\ell_k)} = 0$ and therefore contributes nothing to the objective, so the argument below applies to the non-empty blocks.

We first show that the maximum weighted completion time within a block is attained by the last job of that block. For any $J_j \in \mathcal{B}_k$, the last job of the block completes no earlier than J_j in π, so $C_j \leq C_{\pi(\ell_k)}$, while $w_j \leq L_k$ by the block construction. Hence $w_j C_j \leq L_k C_{\pi(\ell_k)}$, and taking the maximum over $j \in \mathcal{B}_k$ gives

$$\max_{J_j \in \mathcal{B}_k} w_j C_j = L_k C_{\pi(\ell_k)}.$$

The objective of (2) therefore equals

$$WC_{\max}(\pi) = \max_{k=1,\ldots,K} L_k C_{\pi(\ell_k)},$$

so it depends only on the last job of each block.

Let π' be the schedule obtained by rescheduling the jobs within each block $\mathcal{B}_k$ according to Johnson's rule, keeping the blocks in their original order. We prove $C_{\pi'(\ell_k)} \leq C_{\pi(\ell_k)}$ for every k by induction on k.

Base case $k = 1$. Block $\mathcal{B}_1$ starts at time zero on both machines regardless of its internal order, so Johnson's rule gives $C_{\pi'(\ell_1)} \leq C_{\pi(\ell_1)}$.

Inductive step. Assume $C_{\pi'(\ell_{k-1})} \leq C_{\pi(\ell_{k-1})}$. On M_1, the start time of $\mathcal{B}_k$ equals the total M_1 work of $\mathcal{B}_1, \ldots, \mathcal{B}_{k-1}$, which is invariant under internal rescheduling, so $\mathcal{B}_k$ begins at the same instant on M_1 under both schedules. On M_2, block $\mathcal{B}_k$ starts at $\max\{\sum_{i=1}^{k} A_i,\, C_{\pi(\ell_{k-1})}\}$; the first term is identical under π and π', and the second is no larger under π' by the induction hypothesis. Hence $\mathcal{B}_k$ starts no later on M_2 under π' than

under π. Together with the same M_1 start and the internally optimal Johnson scheduling, this gives $C_{\pi'(\ell_k)} \le C_{\pi(\ell_k)}$, which completes the induction.

Finally,

$$WC_{\max}(\pi') = \max_k L_k C_{\pi'(\ell_k)} \le \max_k L_k C_{\pi(\ell_k)} = WC_{\max}(\pi),$$

which establishes the lemma. □

4.2 An exact dynamic program

The exact dynamic program processes the jobs in the Johnson sequence and assigns each job to one of the K blocks $\mathcal{B}_1, \ldots, \mathcal{B}_K$.

Lemma 4.1 provides the structural basis for the dynamic program. Given any assignment of jobs to the K blocks, sequencing the jobs within each block $\mathcal{B}_i$ according to Johnson's rule and concatenating the blocks in the order of non-increasing block weights yields a schedule whose $WC_{\max}$ value is no larger than that of any other schedule respecting the same assignment. For each $j = 1, \ldots, n$, let $\mathscr{H}_j$ denote the set of states attainable after the first j jobs have been processed. A state records the following quantities for every block $\mathcal{B}_i$:

- A_i: the total processing time on machine M_1 of the jobs in block $\mathcal{B}_i$;
- B_i: the total processing time on machine M_2 of the jobs in block $\mathcal{B}_i$;
- C_i: the completion time of the last job of block $\mathcal{B}_i$ on machine M_2;
- L_i: the weight of the last job of block $\mathcal{B}_i$.

Since $W_1 > \cdots > W_K$, the blocks eligible for J_j form a prefix. Writing

$$i^* = \max\{\, i : w_j \le W_i \,\},$$

the eligible set is $\mathcal{E}(J_j) = \{\, 1, \ldots, i^* \,\}$. Each state is therefore the aggregate K-block tuple

$$(A_1, \ldots, A_K;\ B_1, \ldots, B_K;\ C_1, \ldots, C_K;\ L_1, \ldots, L_K),$$

which is described by $4K$ parameters.

The initial state set contains a single state whose components are all zero: $\mathscr{H}_0 = \{(0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0)\}$. In Algorithm 1, T_{A_i} denotes the total M_1 processing time of the first i blocks, and T_{B_i} the completion time of block $\mathcal{B}_i$ on M_2.

Lemma 4.2. *Let $\mathscr{H} = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K)$ and $\mathscr{H}' = (A'_1, \ldots, A'_K; B'_1, \ldots, B'_K; C'_1, \ldots, C'_K; L'_1, \ldots, L'_K)$ be two states in $\mathscr{H}_j$. If $A_i = A'_i$, $B_i = B'_i$ and $L_i = L'_i$ for every i, and $C_i \le C'_i$ for every i, then $\mathscr{H}'$ may be discarded without changing the optimal value.*

Proof. Consider any continuation that extends $\mathscr{H}'$ to a complete schedule Π'. Replacing the prefix represented by $\mathscr{H}'$ with the prefix represented by $\mathscr{H}$ yields a schedule Π that assigns the remaining jobs identically. For every block $\mathcal{B}_k$ that follows the updated one, the M_2 start time is $\max\{\sum_{i=1}^{k} A_i,\, C_{k-1}\}$, and C_{k-1} is no larger under $\mathscr{H}$ than under $\mathscr{H}'$. By induction, the M_2 completion time of every subsequent block is no larger under $\mathscr{H}$. Since A_i, B_i and L_i are identical in the two states, the objective value $WC_{\max}$ of Π does not exceed that of Π'. □

For $j = 1, \ldots, n$, the algorithm extends $\mathscr{H}_{j-1}$ to $\mathscr{H}_j$ by placing J_j at the end of each eligible block. That is, for every state in $\mathscr{H}_{j-1}$ and every $i \in \mathcal{E}(J_j)$, only the i-th slice of the state is updated while all other blocks remain unchanged:

$$A'_i = A_i + a_j, \quad B'_i = B_i + b_j, \quad C'_i = \max\{C_i + b_j,\ A_i + a_j + b_j\}. \tag{25}$$

The resulting candidate states are collected in temporary sets $\mathcal{Y}_1, \ldots, \mathcal{Y}_K$. By Lemma 4.2, merging the candidates removes dominated and duplicate states and yields $\mathscr{H}_j$.

Algorithm 1: Block concatenation and evaluation

Input: State $\mathscr{H} = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K)$.

Step 1. [Initialization] Set $T_{A_0} \leftarrow 0$, $T_{B_0} \leftarrow 0$, $WC_{\max} \leftarrow 0$.
Step 2. [Evaluation] **for** $i = 1$ **to** K **do**
  $T_{A_i} \leftarrow T_{A_{i-1}} + A_i$
  $T_{B_i} \leftarrow \max\{\, T_{A_{i-1}} + C_i,\ T_{B_{i-1}} + B_i \,\}$
  if $L_i > 0$ **then**
    $WC_{\max} \leftarrow \max\{\, WC_{\max},\ L_i \cdot T_{B_i} \,\}$
  end
end

Output: $WC_{\max}$.

Algorithm 2: DP for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j); block count K; weights $W_1 > \cdots > W_K$.

Step 1. [Initialization] Set $L_i \leftarrow 0$ for $i = 1, \ldots, K$, set $\mathscr{H}_0 \leftarrow \{(0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0)\}$ and $Z^* \leftarrow +\infty$.

Step 2. [Generation] **for** $j = 1$ **to** n **do**

  for *each* $(A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K) \in \mathscr{H}_{j-1}$ **do**

    for *each block* $i \in \mathcal{E}(J_j) = \{1, \ldots, i^*\}$ **do**

      $A'_i \leftarrow A_i + a_j$, $B'_i \leftarrow B_i + b_j$, $L'_i \leftarrow w_j$

      $C'_i \leftarrow \max\{\, C_i + b_j,\ A_i + a_j + b_j \,\}$

      Add the state $(A_1, \ldots, A'_i, \ldots, A_K; B_1, \ldots, B'_i, \ldots, B_K; C_1, \ldots, C'_i, \ldots, C_K; L_1, \ldots, L'_i, \ldots, L_K)$ to $\mathscr{H}_j$

    end

  end

end

[Elimination] Elimination $\mathscr{H}_j$ for $j \in \{1, \ldots, n\}$ by Lemma 4.2

Step 3. [Calculate] **for** *each state* $S = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K) \in \mathscr{H}_n$ **do**

  if $L_i \in \{0, W_i\}$ *for every* $i = 1, \ldots, K$ **then**

    $WC_{\max} \leftarrow$ Algorithm 1 applied to S

    $Z^* \leftarrow \min\{\, Z^*,\ WC_{\max} \,\}$

  else

    Discard S.

  end

end

Step 4. [Result] Recover a schedule attaining Z^* by standard backtracking.

Output: An optimal schedule π^*.

To support Step 4, each state in $\mathscr{H}_j$ stores a pointer to the state in $\mathscr{H}_{j-1}$ from which it was derived, together with the block $i \in \mathcal{E}(J_j)$ into which J_j was placed. Starting from a state in $\mathscr{H}_n$ attaining Z^*, backtracking along these pointers recovers the block $\mathcal{B}_i$ containing each job J_j. Sequencing the jobs of each block according to Johnson's rule (Lemma 4.1) and concatenating the blocks then yields an optimal schedule π^* with $WC_{\max}(\pi^*) = Z^*$.

Theorem 4.1. *Algorithm 2 solves* $F2 \parallel WC_{\max}$ *optimally in* $O\big(n \cdot K \cdot |\mathscr{H}|_{\max}\big)$ *time, where* $|\mathscr{H}|_{\max} \le 2^K V^{3K-3}$ *and* $V = \sum_{j=1}^{n} a_j + \sum_{j=1}^{n} b_j$. *For constant* K, *this is* $O(nV^{3K-3})$.

Proof. Each state in $\mathscr{H}_j$ is the $4K$-tuple

$$(A_1, \ldots, A_K;\ B_1, \ldots, B_K;\ C_1, \ldots, C_K;\ L_1, \ldots, L_K),$$

so it suffices to bound the number of distinct values each component can attain.

The quantity A_i is the total M_1 processing time of the jobs assigned to block $\mathcal{B}_i$, hence $A_i \in [0,\ V]$. Since the j jobs considered so far form a prefix of the fixed Johnson sequence,

$\sum_{i=1}^{K} A_i$ equals the sum of the a_h over that prefix. At most $K-1$ of the A_i are therefore independent, contributing at most V^{K-1} distinct A-vectors. Symmetrically, $\sum_{i=1}^{K} B_i$ is the prefix sum of the b_j, so the B_i components likewise contribute at most V^{K-1} distinct combinations.

The quantity C_i is the makespan of $\mathcal{B}_i$ when that block is scheduled in isolation by Johnson's rule. Since $C_i \le A_i + B_i \le V$, the C-vector contributes at most V^{K-1} distinct values. The weight L_i equals W_i when block $\mathcal{B}_i$ is non-empty and 0 when it is empty, so the L-vector takes at most 2^K distinct values. Multiplying these bounds gives

$$|\mathscr{H}_j| \le 2^K \cdot V^{K-1} \cdot V^{2K-2} = 2^K \cdot V^{3K-3}.$$

In iteration j, the algorithm takes each state in $\mathscr{H}_{j-1}$, tries every block $i \in \{1, \dots, K\}$, computes the updated tuple in $O(1)$ time, and inserts it into $\mathcal{Y}_i$. The merge step collects the K sets $\mathcal{Y}_1, \dots, \mathcal{Y}_K$ and removes duplicates in time proportional to the total number of generated tuples. Iteration j therefore costs $O(K \cdot |\mathscr{H}_{j-1}|)$, and summing over the n iterations gives $O(n \cdot K \cdot |\mathscr{H}|_{\max})$.

The final concatenation by Algorithm 1 evaluates each state in $\mathscr{H}_n$ in $O(K)$ time, which is dominated by the DP phase. For fixed K, 2^K is constant and the bound becomes $O(n \cdot V^{3K-3})$. □

4.3 A 2-approximation

The exact dynamic program of Section 4 runs in pseudo-polynomial time, and the FPTAS of Section 4.4 has running time exponential in the number K of distinct weights. When only a constant-factor guarantee is needed, the following elementary rule suffices.

Algorithm 3: 2-Approximation for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j).

1 **Step 1.** Sort the jobs by non-increasing weight w_j, breaking ties by non-increasing $a_j + b_j$.

2 **Step 2.** Schedule the jobs in this sequence on both machines.

Output: The resulting permutation schedule π.

Theorem 4.2. *Algorithm 3 runs in $O(n \log n)$ time and returns a schedule with $WC_{\max} \le 2\, WC^*_{\max}$.*

Proof. Let π be the schedule returned by Algorithm 3. Its jobs appear in non-increasing order of weight, so we name them $J_1, J_2, \dots, J_n$, with $w_1 \ge w_2 \ge \cdots \ge w_n$ and $\pi = (J_1, J_2, \dots, J_n)$. Every job of the prefix $S = \{J_1, J_2, \dots, J_j\}$ then carries a weight of at least w_j.

We first bound the completion times in π. By (1), the job J_j, which sits at position j of π, completes on M_2 at time $C_{\pi(j)}$, and every term of that maximum is bounded by the full

sum:

$$\sum_{h=1}^{i} a_h + \sum_{h=i}^{j} b_h \le \sum_{h=1}^{j} a_h + \sum_{h=1}^{j} b_h = \sum_{h\in S} \big(a_h + b_h\big), \qquad 1 \le i \le j.$$

Hence

$$C_{\pi(j)} \;\le\; \sum_{h\in S} \big(a_h + b_h\big). \tag{26}$$

Since $WC_{\max}(\pi) = \max_{1\le j\le n} w_{\pi(j)}\, C_{\pi(j)}$ ((2)), and $w_{\pi(j)} = w_j$ for the job at position j, multiplying (26) by $w_j > 0$ and taking the maximum over j gives

$$WC_{\max}(\pi) = \max_{1\le j\le n} w_{\pi(j)}\, C_{\pi(j)} \le \max_{1\le j\le n}\; w_j \sum_{h\in S} \big(a_h + b_h\big). \tag{27}$$

It remains to bound the right-hand side by $2WC^*_{\max}$. Fix a prefix $S = \{J_1, J_2, \ldots, J_j\}$ and let π^* be an optimal schedule. Three quantities bound the optimum from below,

$$LB_1 = \max_{1\le j\le n} w_j \sum_{h\in S} a_h, \tag{28}$$

$$LB_2 = \max_{1\le j\le n} w_j \sum_{h\in S} b_h, \tag{29}$$

$$LB_3 = \max_{1\le j\le n} w_j\,(a_j + b_j), \tag{30}$$

the M_1 threshold LB_1, the M_2 threshold LB_2 and the job threshold LB_3, and their largest is the reference value of the instance,

$$LB = \max\{LB_1, LB_2, LB_3\}, \tag{31}$$

so that $LB \le WC^*_{\max}$.

To see this, let J_u be the job of S that finishes last on M_1 in π^*, and J_v the job of S that finishes last on M_2. All jobs of S are processed on M_1 before J_u completes M_1, so $C^*_u \ge \sum_{h\in S} a_h$. In the same way, all M_2 processing of the jobs of S precedes the completion of J_v on M_2, so $C^*_v \ge \sum_{h\in S} b_h$. Since both J_u and J_v lie in S, their weights satisfy $w_u, w_v \ge w_j$. The value $WC^*_{\max}$ bounds the weighted completion time of every job of π^*, so $WC^*_{\max} \ge w_u C^*_u \ge w_j \sum_{h\in S} a_h$ and $WC^*_{\max} \ge w_v C^*_v \ge w_j \sum_{h\in S} b_h$. Every job J_j of S is processed on M_1 before it is processed on M_2, and neither operation can be left out, so $C^*_j \ge a_j + b_j$ and $WC^*_{\max} \ge w_j C^*_j \ge w_j(a_j + b_j)$. The prefix S was arbitrary, so these bounds hold for every prefix.

For the prefix S, the first two bounds give $w_j \sum_{h\in S}(a_h + b_h) \le LB_1 + LB_2 \le 2\,LB \le 2\,WC^*_{\max}$, and the claim follows from (27). □

4.4 A fully polynomial-time approximation scheme

4.4.1 Rounding the time coordinates

The exact dynamic program of Algorithm 2 becomes a fully polynomial-time approximation scheme once the input data are rounded (Hall, 1994). The technique rests on bounds for the coordinates that are rounded: each time coordinate ranges over a bounded interval, and rounding it to one of a fixed number of geometrically spaced values keeps the relative error within a factor θ. The total processing times A_i, B_i of block $\mathcal{B}_i$ on M_1 and M_2, and its completion time C_i, are rounded. The weight L_i of the last job of block $\mathcal{B}_i$ is not rounded; following Algorithm 2, the positional constraint $L_i \in \{0, W_i\}$ is required for every block, and a state violating it is discarded.

Fix $0 < \epsilon < 1$ and let

$$\theta = 1 + \frac{\epsilon}{2n}, \qquad V_A = \sum_{j=1}^{n} a_j, \qquad V_B = \sum_{j=1}^{n} b_j, \qquad V = V_A + V_B.$$

For every block $\mathcal{B}_i$, the recurrence (25) preserves $A_i \le V_A$ and $B_i \le V_B$, since block $\mathcal{B}_i$ contains a subset of the jobs; moreover $C_i \le A_i + B_i \le V$. We therefore round the A dimension over $[0, V_A]$, the B dimension over $[0, V_B]$, and the C dimension over $[0, V]$. To this end, let

$$\Delta(x) = k = \lfloor \log_\theta x \rfloor \quad \text{for every } x \ge 1 \text{ with } \theta^k \le x < \theta^{k+1}, \qquad \text{and} \qquad \Delta(0) = -1.$$

All processing times are integral, so every value taken by a coordinate A_i, B_i, C_i is a non-negative integer. Hence any two values $y, y' \ge 1$ with $\Delta(y) = \Delta(y')$ satisfy $y \le \theta\, y'$; we call this the *interval-dominance* property, and it is the only fact about the rounding used in the analysis below.

The *label* of a state $\mathscr{H}^\epsilon = (A_1, \dots, A_K; B_1, \dots, B_K; C_1, \dots, C_K; L_1, \dots, L_K)$ is the tuple formed by applying Δ to its time coordinates, together with its block-end weights,

$$\ell(\mathscr{H}^\epsilon) = \big(\Delta(A_1), \dots, \Delta(A_K);\ \Delta(B_1), \dots, \Delta(B_K);\ \Delta(C_1), \dots, \Delta(C_K);\ L_1, \dots, L_K\big).$$

Thus two states share a label exactly when every coordinate A_i, B_i and C_i falls into the same box and their block-end weights coincide. Each dimension $x \in \{A, B, C\}$ contributes at most $v^x + 2$ distinct values of Δ, where $v^x = \lceil \log_\theta V_x \rceil$ and $V_C = V$, while the weight is not rounded and each L_i takes at most $n + 1$ values.

The Elimination step keeps one representative per label. At iteration j, every state of $\mathscr{H}^\epsilon_{j-1}$ spawns one candidate for each eligible block $i \in \mathcal{E}(J_j) = \{1, \dots, i^*\}$, and each candidate is assigned its label. Two candidates with the same label are indistinguishable after rounding and carry the same block-end weights $L_1, \dots, L_K$, so the step keeps any one of them and discards the rest: the kept state satisfies $x \le \theta\, \tilde{x}$ in every time coordinate

$x \in \{A, B, C\}$ of every block, relative to every candidate $\tilde{\mathscr{H}}^{\epsilon}$ it replaces, while its block-end weights are unchanged. Every label therefore carries exactly one state into $\mathscr{H}_j^{\epsilon}$, and the number of states never exceeds the number of labels.

Algorithm 4: State-class FPTAS for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j); block count K; accuracy ϵ.

Step 1. [Initialization] Set $\theta = 1 + \epsilon/(2n)$; set $Z^* \leftarrow +\infty$;
$\mathscr{H}_0^{\epsilon} \leftarrow \{(0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0)\}$.
Step 2. [Generation] **for** $j = 1$ **to** n **do**
  for *each state* $(A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K) \in \mathscr{H}_{j-1}^{\epsilon}$ **do**
    for *each block* $i \in \mathcal{E}(J_j) = \{1, \ldots, i^*\}$ **do**
      $\tilde{A}_i = A_i + a_j$, $\tilde{B}_i = B_i + b_j$, $\tilde{L}_i = w_j$
      $\tilde{C}_i = \max\{\, C_i + b_j,\ A_i + a_j + b_j \,\}$
      [Label] Record the label $\ell(\tilde{\mathscr{H}}_j^{\epsilon}) =$
       $\big(\Delta(\tilde{A}_1), \ldots, \Delta(\tilde{A}_K);\ \Delta(\tilde{B}_1), \ldots, \Delta(\tilde{B}_K);\ \Delta(\tilde{C}_1), \ldots, \Delta(\tilde{C}_K);\ \tilde{L}_1, \ldots, \tilde{L}_K\big)$.
    end
    [Elimination] Among the candidate states sharing the same label, keep one of them and delete the others; set $\mathscr{H}_j^{\epsilon}$ to the kept states.
  end
end
Step 3. [Calculate] Same as Step 3 of Algorithm 2, applied to $\mathscr{H}_n^{\epsilon}$.
Step 4. [Result] Same as Step 4 of Algorithm 2.
Output: An approximate schedule π.

Lemma 4.3. *Let* $\mathscr{H}_j = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K)$, $j = 1, \ldots, n$, *be the states along an optimal trajectory of Algorithm 2, the states that backtracking from an optimal final state recovers. For every* j, *Algorithm 4 keeps a state* $\hat{\mathscr{H}}_j^{\epsilon} = (\hat{A}_1, \ldots, \hat{A}_K; \hat{B}_1, \ldots, \hat{B}_K; \hat{C}_1, \ldots, \hat{C}_K; L_1, \ldots, L_K)$ *that carries the same block-end weights* $L_1, \ldots, L_K$ *and satisfies*

$$\hat{A}_i \le A_i\, \theta^{j-1}, \quad \hat{B}_i \le B_i\, \theta^{j-1}, \quad \hat{C}_i \le C_i\, \theta^{j-1}, \quad (1 \le i \le K). \tag{32}$$

Proof. We argue by induction on j, from 1 to n.

Base case $j = 1$**.** Both algorithms generate their candidates for J_1 from the same zero state. Two candidates that place J_1 in different blocks differ in at least one time coordinate, so they carry different labels and no state is deleted. The sets $\mathscr{H}_1^{\epsilon}$ and $\mathscr{H}_1$ therefore coincide, and (32) holds with equality at $j = 1$.

Inductive step $j \ge 2$**.** Assume that (32) holds at $j - 1$, and let i be the block to which $\mathscr{H}_{j-1}$ appends J_j in Algorithm 2. Algorithm 2 assigns a job only to a block i with $w \le W_i$, so $w_j \le W_i$. This constraint depends only on J_j and the block index, hence appending J_j to block i is available from $\hat{\mathscr{H}}_{j-1}^{\epsilon}$ as well. Write A_i', B_i' and C_i' for the coordinates of $\mathscr{H}_{j-1}$.

Applying the exact recurrences to $\hat{\mathscr{H}}^{\epsilon}_{j-1}$ and using $\theta^{j-2} \geq 1$ gives a candidate $\tilde{\mathscr{H}}^{\epsilon}_{j}$ with

$$\begin{aligned}
\tilde{A}_i &= \hat{A}_i + a_j \leq \big(A'_i + a_j\big)\theta^{j-2} = A_i\theta^{j-2},\\
\tilde{B}_i &= \hat{B}_i + b_j \leq \big(B'_i + b_j\big)\theta^{j-2} = B_i\theta^{j-2},\\
\tilde{C}_i &= \max\big\{\hat{C}_i + b_j,\ \hat{A}_i + a_j + b_j\big\}\\
&\leq \max\big\{C'_i + b_j,\ A'_i + a_j + b_j\big\}\theta^{j-2} = C_i\theta^{j-2},
\end{aligned}$$

where A_i, B_i and C_i are the coordinates of $\mathscr{H}_j$. For the remaining blocks $m \neq i$ the same transition leaves the coordinates unchanged, so that $\tilde{x}_m = \hat{x}_m \leq x'_m\theta^{j-2} = x_m\theta^{j-2}$ for $x \in \{A, B, C\}$. Hence $\tilde{x}_m \leq x_m\theta^{j-2}$ holds for every block $m = 1, \ldots, K$. The block-end weights $L_1, \ldots, L_K$ of the candidate are those of $\mathscr{H}_j$, because $L_i = w_j$ and the remaining block-end weights are unchanged.

The candidate $\tilde{\mathscr{H}}^{\epsilon}_{j}$ carries the label $\ell(\tilde{\mathscr{H}}^{\epsilon}_{j})$ assigned in Algorithm 4, and the algorithm keeps a state $\hat{\mathscr{H}}^{\epsilon}_{j} \in \mathscr{H}^{\epsilon}_{j}$ with that same label. Since the label records the block-end weights, $\hat{\mathscr{H}}^{\epsilon}_{j}$ carries $L_1, \ldots, L_K$ as well. Interval dominance states that two values with the same index Δ differ by at most a factor θ, so $y^x_m \leq \theta\,\tilde{x}_m$ for every $x \in \{A, B, C\}$ and every block $m = 1, \ldots, K$, where y^x_m denotes the coordinates of $\hat{\mathscr{H}}^{\epsilon}_{j}$. Together with $\tilde{x}_m \leq x_m\theta^{j-2}$, this yields (32) at j and completes the induction.

Every state of $\mathscr{H}^{\epsilon}_{n}$ stores all $3K$ time coordinates, so the induction covers the coordinates of the last block $\mathcal{B}_K$ as well. Storing these coordinates for every block, exactly as in Algorithm 2, rather than deriving them from the others, preserves the factor θ^{j-1} of (32): a derived value would accumulate the rounding errors of the remaining blocks. □

Theorem 4.3. *For every $0 < \epsilon < 1$, Algorithm 4 returns a schedule with*

$$WC_{\max} \ \leq\ (1+\epsilon)\,WC^*_{\max}.$$

For fixed K, it runs in $O\Big(n^{4K+1} \cdot K \cdot \frac{(\log V)^{3K}}{\epsilon^{3K}}\Big)$ time, which is polynomial in n, $\log V$ and $1/\epsilon$.

Proof. We first bound the value of the returned schedule, and then its running time.

Let $\mathscr{H}_n$ be the final state of the optimal trajectory of Algorithm 2, and let $\hat{\mathscr{H}}^{\epsilon}_{n} \in \mathscr{H}^{\epsilon}_{n}$ be the state guaranteed by Lemma 4.3 at $j = n$. By the lemma, the time coordinates of $\hat{\mathscr{H}}^{\epsilon}_{n}$ are within a factor θ^{n-1} of those of $\mathscr{H}_n$, and its block-end weights are those of $\mathscr{H}_n$, because the weights are not rounded. The final state $\mathscr{H}_n$ satisfies the positional constraint $L_i \in \{0, W_i\}$ of Algorithm 2, so every block of $\hat{\mathscr{H}}^{\epsilon}_{n}$ has block-end weight at most W_i.

Let $\hat{\pi}$ and π^* be the schedules obtained by concatenating the blocks of $\hat{\mathscr{H}}^{\epsilon}_{n}$ and $\mathscr{H}_n$, and let $\hat{C}_i$ and C_i denote the completion times of the last job of block $\mathcal{B}_i$ in the two schedules, which are the corresponding coordinates of the two states. By the concatenation recurrence of Algorithm 1, C_i is nondecreasing in every coordinate of the state and positively homogeneous.

Hence $\hat{C}_i \le \theta^{n-1} C_i$ for every i. Since every block of $\hat{\mathscr{H}}_n^{\epsilon}$ has block-end weight at most W_i, Lemma 4.1 yields

$$WC_{\max}(\hat{\pi}) = \max_{1\le i\le K} W_i \hat{C}_i \le \theta^{n-1} \max_{1\le i\le K} W_i C_i = \theta^{n-1} WC^*_{\max}.$$

The factor θ^{n-1} is at most $1+\epsilon$ for $0<\epsilon<1$. Because $1+x\le e^x$ for every real x,

$$\theta^{n-1} = \left(1+\frac{\epsilon}{2n}\right)^{n-1} \le e^{\frac{(n-1)\epsilon}{2n}} < e^{\epsilon/2},$$

where the strict inequality uses $(n-1)/(2n) < 1/2$. It is therefore enough to check that $e^{\epsilon/2} \le 1+\epsilon$ on $[0,1]$. The function $g(\epsilon) = 1+\epsilon-e^{\epsilon/2}$ satisfies $g(0)=0$ and

$$g'(\epsilon) = 1 - \tfrac{1}{2}e^{\epsilon/2} \ge 1 - \tfrac{1}{2}e^{1/2} > 0 \qquad (0\le\epsilon\le 1),$$

because $e^{\epsilon/2} \le e^{1/2} < 2$ on that interval. Hence g is increasing, so $g(\epsilon)\ge 0$ and $e^{\epsilon/2}\le 1+\epsilon$. Consequently $\theta^{n-1}\le 1+\epsilon$, and the schedule returned by Algorithm 4, being no worse than $\hat{\pi}$, has a value of at most $(1+\epsilon)\,WC^*_{\max}$.

We now bound the running time. At each level j, Algorithm 4 keeps at most one state per label, and the number of labels is $(v^A+2)^K(v^B+2)^K(v^C+2)^K(n+1)^K$. The rounded time coordinates contribute $(v^A+2)^K(v^B+2)^K(v^C+2)^K$, which is at most $(v^C+2)^{3K}$ since $v^A, v^B \le v^C$, and each of the K block-end weights takes at most $n+1$ values. Each state generates at most K candidates, one per eligible block, in $O(1)$ time each. The final evaluation of the states of $\mathscr{H}^{\epsilon,n}$ costs $O(K)$ per state. The running time is therefore $O\big(nK\,(v^C+2)^{3K}(n+1)^K\big)$. Since $\ln(1+\epsilon/(2n)) \ge \epsilon/(4n)$ for $0<\epsilon<1$, we have

$$v^C = \left\lceil \frac{\ln V}{\ln\theta} \right\rceil = O\left(\frac{n\log V}{\epsilon}\right).$$

For constant K, the bound reduces to $O(n^{4K+1}K(\log V)^{3K}/\epsilon^{3K})$, which is polynomial in n, $\log V$ and $1/\epsilon$. □

4.4.2 Rounding the weights

The FPTAS above rounds the time coordinates A_i, B_i, C_i and assumes that the number K of distinct weights is fixed. An analogous geometric rounding applies to the weights themselves when their ratio is bounded. Let $W_{\min} = \min_j w_j$, $W_{\max} = \max_j w_j$, and suppose $W_{\max}/W_{\min} \le c$ for a constant c. Partition the interval $[W_{\min}, W_{\max}]$ into

$$J_0 = [W_{\min}, W_{\min}\theta), \qquad J_r = [W_{\min}\theta^r, W_{\min}\theta^{r+1}) \ \ (1\le r\le q-1), \qquad J_q = [W_{\min}\theta^q, W_{\max}],$$

with $q = \lceil \log_\theta(W_{\max}/W_{\min}) \rceil$, and round each weight w_j upward to the right endpoint of its interval,

$$\hat{w}_j = W_{\min}\,\theta^{\,r_j+1}, \qquad r_j = \left\lfloor \log_\theta \frac{w_j}{W_{\min}} \right\rfloor,$$

so that $w_j \le \hat{w}_j \le \theta\, w_j$. The rounded weights take only $q+1 = O\big(\log_\theta(W_{\max}/W_{\min})\big)$ distinct values, so jobs whose weights differ by a factor at most θ merge into a single class.

Lemma 4.4. *Let $\hat{w}_j$ be the rounded weights defined above, and let $\hat{\pi}$ be an optimal schedule of the instance with weights $\hat{w}_j$, whose optimal value is $\hat{W}C^*_{\max}$. Then*

$$WC_{\max}(\hat{\pi}) \leq \theta\, WC^*_{\max}. \tag{33}$$

Proof. By construction $w_j \leq \hat{w}_j \leq \theta w_j$ for every job J_j. By (2) and the monotonicity of the maximum, every schedule π satisfies

$$WC_{\max}(\pi) = \max_{1\leq j\leq n} w_{\pi(j)}C_{\pi(j)} \leq \max_{1\leq j\leq n} \hat{w}_{\pi(j)}C_{\pi(j)} = \hat{W}C_{\max}(\pi) \leq \theta\, WC_{\max}(\pi).$$

Let π^* be an optimal schedule of the original instance. The rounded problem is solved optimally by $\hat{\pi}$, and π^* is feasible for it, so

$$WC_{\max}(\hat{\pi}) \leq \hat{W}C_{\max}(\hat{\pi}) = \hat{W}C^*_{\max} \leq \hat{W}C_{\max}(\pi^*) \leq \theta\, WC_{\max}(\pi^*) = \theta\, WC^*_{\max},$$

which is (33). □

Hence the weight rounding incurs a relative error of at most $\theta = 1 + \epsilon/(2n) \leq 1 + \epsilon$, exactly as in the completion-time rounding.

5 Heuristic algorithms

This section presents two heuristic methods for instances that are too large to be solved efficiently by the exact methods: an adaptation of the NEH heuristic, which constructs a schedule through sequential job insertion, and a genetic algorithm, which searches the space of job permutations.

5.1 The NEH heuristic

The NEH heuristic constructs a sequence by inserting jobs one at a time. Since a long job delays the completion of all jobs scheduled after it on both machines, jobs are considered in non-increasing order of $a_j + b_j$, with ties broken by non-increasing order of w_j. Each job is then inserted into the position that yields the smallest objective value. The detailed procedure is presented in Algorithm 5. Evaluating all k possible insertion positions at iteration k requires $O(k^2)$ time using (1). Summing over all iterations gives an overall running time of $O(n^3)$, the same complexity as that of the original NEH heuristic; NEH-based constructions for the permutation flow shop have since refined this insertion scheme (Puka et al., 2024).

Algorithm 5: NEH heuristic for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j).

Step 1. [Sequence] Sort the jobs by non-increasing $a_j + b_j$, breaking ties by w_j; let $J_1, \ldots, J_n$ be the resulting sequence.

Step 2. [Initialization] $\pi \leftarrow (J_1)$.

Step 3. [Insertion] **for** $k = 2$ **to** n **do**

  $\pi \leftarrow$ the sequence obtained by inserting J_k into π at a position minimizing $WC_{\max}$.

end

Output: Schedule π.

5.2 The genetic algorithm

The objective is a maximum, so the value of a schedule is determined by its job with the largest weighted completion time. The search must therefore reduce the weighted completion time of the critical job without causing any other job's weighted completion time to exceed it. Since the weight-based and processing-time-based ordering criteria may conflict, the jobs cannot be treated symmetrically.

The algorithm adopts the permutation-based framework of Qiao et al. (2022), following their applications in recent flow-shop studies (Wang et al., 2023). Specifically, it employs a population initialized with Johnson's schedule, binary tournament selection, partially mapped crossover (Figure 4), and non-dominated sorting with crowding distance. Guided by Lemma 4.1, the mutation operator moves an entire block of jobs sharing the same weight rather than a single job. A subsequent local search reinserts the jobs with the largest weighted completion times. The complete procedure is presented in Algorithm 6.

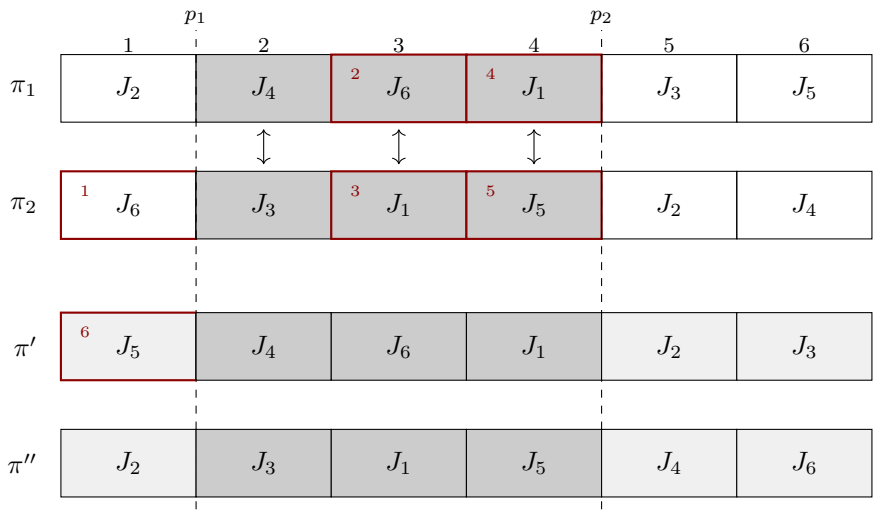


Figure 4: One application of the partially mapped crossover, with $p_1 = 2$ and $p_2 = 4$

5.2.1 Encoding and decoding

A chromosome is a permutation of the jobs, read by both machines in the same order, so crossover and mutation always return feasible schedules. Its objective follows from (1) and (2) in linear time; Figure 5 gives an example.

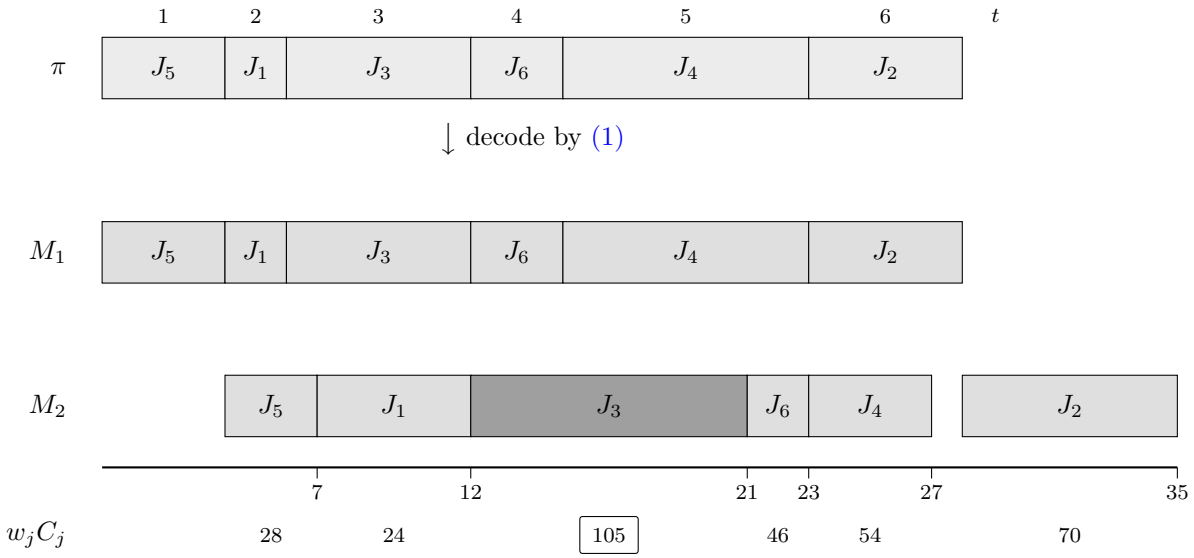


Figure 5: Encoding and decoding

Algorithm 6: Genetic Algorithm for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$; population size N; generations G; offspring λ; probabilities p_c, p_m; reinsertion count k.

Step 1. [Initialization] Build P_0 from Johnson's schedule and $N-1$ neighbours of it; evaluate all individuals; set π^* to the best.

Step 2. [Evolution] **for** $g = 1$ **to** G **do**

  Selection Form the mating pool M_g from P_{g-1} by binary tournament selection.

  Crossover while $|O_g| < \lambda$ **do**

    Draw two parents from M_g; with probability p_c apply the crossover to obtain two children, otherwise copy the parents into the offspring set O_g.

  end

  Mutation Apply the block mutation to each offspring with probability p_m; evaluate every child by (1).

  Survival $P_g \leftarrow$ the N survivors of $P_{g-1} \cup O_g$, taken by non-dominated sorting on the three objectives and, on the last front, by crowding distance.

  Elite If a child of O_g improves on $WC_{\max}(\pi^*)$, replace π^* by it, and if π^* is not among the survivors, let it take the place of the one the ranking leaves last, so that it is a parent of generation $g+1$.

end

Step 3. [Local search] **repeat**

  Let $J_1, \ldots, J_k$ be the jobs of π^* with the largest $w_j C_j$.

  Form, for every J_i and every position r, the schedule obtained by moving J_i to position r, and evaluate it by (1); let π' be the best of them.

  if $WC_{\max}(\pi') < WC_{\max}(\pi^*)$ **then**

    $\pi^* \leftarrow \pi'$

  end

until *no reinsertion of the step improves*

Output: Best schedule π^*.

6 Experiments

All algorithms were implemented in Python 3.13 and tested on a machine equipped with an Intel Core Ultra 5 230F processor and 16 GB RAM running Windows 11. The MILP model was solved using Gurobi 13.0 with a single thread. Each method was executed once per instance under a common time limit of 600 s, and all objective values were evaluated using the same recurrence (1) to ensure a consistent comparison. The dynamic programming (DP) algorithm, the only exact method considered, was evaluated by the number of instances solved to optimality and its running time. The remaining methods were assessed in terms of their relative percentage gap and computational time, where

$$\mathrm{gap}_A = \frac{z_A - z_{\mathrm{base}}}{z_{\mathrm{base}}} \times 100\%, \tag{34}$$

with z_A denoting the objective value obtained by method A and z_{base} the reference value. Both metrics are summarized by their mean and maximum across the instances in each configuration. For small instances, the value returned by the MILP model serves as the reference, which is the optimum wherever the model closes the instance, whereas for large instances, the lower bound $LB = \max\{LB_1, LB_2, LB_3\}$ defined in (31) is used. Thus, the large-instance gaps are independent of the solutions produced by the competing methods.

Three instance groups were generated using integer-valued normal distributions, with the mean at the midpoint of the specified range, a standard deviation equal to one-sixth of its width, and rounded draws clamped to the range. The first group evaluates the DP algorithm, considering $n \in \{40, 60, 80, 100\}$, $K = 2$, weights $\{1, 10\}$, and processing-time ranges $[1, 40]$ and $[1, 80]$. The weight set is extended geometrically, with a growth factor of 20, until the time limit is reached. All instances in the first range are solved within the limit, whereas six of the twenty instances with $n = 100$ in the second range remain unsolved. This group is reported separately because the DP does not scale to the larger instances considered in the algorithmic comparison.

The remaining two groups are designed to compare the MILP model and heuristic methods. Both use the same two processing-time ranges, while the number of distinct weights is at most 30, with weights independently drawn from $\{1, \ldots, 30\}$. Processing times are independently generated for each job using the same distribution. The small-instance group considers $n = 10, 15, \ldots, 60$, allowing the MILP model to provide the reference value, while the large-instance group considers $n = 100, 120, \ldots, 400$, where all heuristic gaps are measured against LB. Each configuration contains twenty independently generated instances. Methods reaching the time limit are evaluated using their best solutions obtained within the available budget.

All reported results are summarized by the mean (avg) and maximum (max) across the twenty instances, with gaps expressed as percentages and running times in milliseconds. Scientific notation is used for large running times, and opt. denotes the number of instances solved to optimality by the DP algorithm. The smallest mean gap in each row is highlighted in bold based on unrounded values. For concise presentation, the MILP formulation, DP algorithm, the 2-approximation and NEH heuristics, and the genetic algorithm are subsequently referred to as the model, program, rules, and search, respectively. The experiments address four questions: the scalability of the DP algorithm, the effectiveness and computational cost of the approximation rules, the performance gains and computational overhead of the genetic algorithm, and the remaining optimality gap indicated by the lower bound.

6.1 The dynamic program

The dynamic programming (DP) algorithm is the only exact method considered in this study and guarantees optimality whenever it terminates. It solves all twenty instances in

seven of the eight configurations, with mean running times of 1.47 s and 4.98 s for forty-job instances under processing-time ranges of [1,40] and [1,80], respectively (Table 1). However, its scalability is limited by both the number of distinct weight classes and the instance size: the state space grows rapidly with both (Section 4), and a third weight class exhausts the available memory before the search starts. This explains why the DP experiments are restricted to $K = 2$.

Instance size and processing-time range further affect computational performance through V. For processing times in [1,40], the mean running time increases from 1.47 s for forty jobs to 89.59 s for one hundred jobs, a 61-fold increase, whereas the mean objective value grows by only 2.5 times. Extending the processing-time range to [1,80] further increases the running time by factors of 2.9–3.7 across the tested instance sizes. The computational limit is reached for one hundred jobs under [1,80], where six of the twenty instances cannot be solved within the 600 s budget. Among the fourteen completed instances, the mean and maximum running times are 329.01 s and 628.50 s, respectively; the latter exceeds the nominal time limit because the termination condition is checked only once per job. Overall, the DP algorithm is computationally effective for small instances but becomes impractical as the instance size and processing-time range increase, whereas the heuristic methods remain applicable to instances with up to 400 jobs.

Table 1: The dynamic program, $K = 2$

n	$a_j, b_j \in [1, 40]$					$a_j, b_j \in [1, 80]$				
	$WC_{\max}$		time (ms)		opt.	$WC_{\max}$		time (ms)		opt.
	avg	max	avg	max		avg	max	avg	max	
40	4335.0	5480.0	1.5e3	4.3e3	20/20	8604.5	10590.0	5.0e3	2.4e4	20/20
60	6340.0	7270.0	1.4e4	3.1e4	20/20	12344.5	15020.0	5.3e4	1.1e5	20/20
80	8221.0	9750.0	4.5e4	1.1e5	20/20	17341.5	19580.0	1.3e5	2.8e5	20/20
100	10647.0	12350.0	9.0e4	1.7e5	20/20	21544.3	24240.0	3.3e5	6.3e5	14/20

Averages at $n = 100$ on $[1, 80]$ cover the 14 instances settled within the 600 s limit; the other six are cut short.

6.2 The small scale

The MILP model solves 436 of the 440 small-scale instances to optimality and provides the reference values for all instances solved to optimality. Its computational cost increases substantially with instance size: the mean running time rises from 70 ms to 160.7 s under [1,40] and from 50.6 ms to 192.0 s under [1,80] as the number of jobs increases from 10 to 60. At the largest instance size, four instances reach the 600-s time limit without being solved to optimality. The processing-time range has no consistent effect on computational time, with the wider range requiring less time in only three of the eleven instance sizes. However, it substantially increases the mean objective value, from 2,607.8 to 5,029.9 for 10 jobs and

from 13,051.7 to 26,079.5 for 60 jobs.

NEH consistently outperforms the 2-approximation across all 22 configurations, achieving a mean gap of 3.14% compared with 5.77% for the latter. Their worst-instance gaps are 24.89% and 21.49%, respectively, occurring at 10 and 25 jobs. The 2-approximation, however, offers a substantial computational advantage, with a mean running time of 0.02 ms and a maximum of 0.05 ms, approximately two orders of magnitude lower than the 2.66-ms average of NEH. It is therefore an attractive alternative when computational efficiency is more important than the improvement in solution quality offered by NEH.

The genetic algorithm (GA) exhibits a different performance trend. Its mean gap decreases from 12.61% to 1.70% under [1,40] and from 14.29% to 3.90% under [1,80] as the number of jobs increases from 10 to 60. At 60 jobs, it achieves the best solution quality among the three heuristics, although its overall mean gap of 10.07% remains considerably higher than those of the 2-approximation and NEH. The improvement is not monotonic: the worst configuration-level mean gap is 18.21% at 15 jobs, while the worst individual-instance gap reaches 57.53% at 25 jobs. This improvement in solution quality comes at a higher computational cost, with the mean running time increasing from 0.64 ms for 10 jobs to 1.69 s for 60 jobs. Nevertheless, the GA remains substantially faster than the MILP model at the largest instance size. Its relatively weak performance on smaller instances, together with its substantial improvement on larger ones, motivates a further evaluation on large-scale instances. The results for both processing-time ranges are reported in Tables 2 to 3, while Figure 6 illustrates the corresponding gap trends.

Overall, at the largest small-instance size, the three heuristics achieve comparable solution quality, with their mean gaps differing by less than three percentage points under both processing-time ranges. The GA progressively closes the gap with the constructive heuristics as the instance size increases, although this improvement comes at a substantially higher computational cost, with running times differing by up to five orders of magnitude. Among the constructive methods, NEH generally produces better schedules, whereas the 2-approximation offers a more favorable trade-off in computational efficiency. Whether the additional computational effort required by the GA translates into further improvements in solution quality on larger instances is investigated next.

Table 2: The small scale, $a_j, b_j \in [1, 40]$: MILP, 2-approx., NEH, GA

n	MILP				2-approx.				NEH				GA			
	$WC_{\max}$		time (ms)		gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
10	2607.8	4369.0	7.0e1	1.2e2	7.3	17.2	8.8e−3	2.5e−2	**3.4**	24.9	7.3e−2	7.9e−2	12.6	30.6	6.4e−1	7.2e−1
15	3758.2	5049.0	1.9e2	4.7e2	6.8	16.9	9.9e−3	1.9e−2	**2.1**	8.5	1.9e−1	2.0e−1	18.2	49.5	2.6e0	2.9e0
20	4811.3	5967.0	5.6e2	1.9e3	6.2	19.9	1.3e−2	2.3e−2	**2.7**	11.7	3.8e−1	4.1e−1	12.1	33.0	1.0e1	1.1e1
25	5590.8	7059.0	1.5e3	5.7e3	5.6	21.5	1.4e−2	2.5e−2	**3.7**	17.5	7.0e−1	7.4e−1	15.3	57.5	2.8e1	3.3e1
30	6888.1	8025.0	2.8e3	1.0e4	5.6	14.9	1.4e−2	2.6e−2	**3.9**	10.8	1.1e0	1.2e0	11.6	26.7	6.2e1	7.0e1
35	7719.7	9373.0	8.4e3	3.6e4	5.7	12.2	1.4e−2	2.4e−2	**3.2**	13.8	1.8e0	1.8e0	12.8	38.7	1.2e2	1.3e2
40	8830.8	10309.0	2.1e4	1.2e5	4.9	10.6	1.7e−2	3.0e−2	**2.4**	6.2	2.5e0	2.6e0	7.0	17.5	2.4e2	2.5e2
45	9669.2	11791.0	3.1e4	2.5e5	6.0	19.0	1.8e−2	3.8e−2	**3.7**	10.3	3.5e0	3.7e0	8.8	24.7	4.1e2	4.5e2
50	10904.9	12558.0	8.4e4	5.1e5	4.5	13.0	1.8e−2	3.1e−2	**3.6**	8.4	4.8e0	4.9e0	6.5	22.9	6.7e2	7.3e2
55	11858.3	14340.0	1.1e5	5.2e5	4.7	10.9	2.7e−2	4.7e−2	**3.1**	11.9	6.2e0	6.4e0	4.2	14.6	1.0e3	1.2e3
60	13051.7	15330.0	1.6e5	6.0e5	4.7	11.7	2.5e−2	4.2e−2	4.3	8.8	8.0e0	8.2e0	**1.7**	8.2	1.7e3	1.9e3

Table 3: The small scale, $a_j, b_j \in [1, 80]$: MILP, 2-approx., NEH, GA

n	MILP				2-approx.				NEH				GA			
	$WC_{\max}$		time (ms)		gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
10	5029.9	7345.0	5.1e1	1.0e2	8.1	17.5	8.5e−3	1.8e−2	**0.8**	5.7	7.2e−2	7.8e−2	14.3	47.5	6.1e−1	6.8e−1
15	7182.1	8624.0	2.9e2	1.8e3	7.7	19.7	9.5e−3	2.0e−2	**2.4**	14.5	1.9e−1	2.1e−1	15.9	37.5	2.6e0	2.9e0
20	9328.9	12103.0	4.5e2	1.8e3	4.5	11.8	1.2e−2	2.9e−2	**2.6**	9.6	3.8e−1	4.1e−1	16.3	39.1	1.0e1	1.2e1
25	10816.3	14100.0	1.6e3	4.6e3	4.7	16.2	1.2e−2	2.1e−2	**2.1**	7.9	7.0e−1	7.4e−1	15.7	37.6	2.8e1	3.0e1
30	12972.5	16005.0	3.2e3	9.5e3	5.6	11.7	1.5e−2	2.9e−2	**2.6**	13.6	1.2e0	1.2e0	14.4	34.2	6.1e1	6.6e1
35	15093.3	17667.0	8.7e3	4.1e4	7.0	18.1	1.8e−2	3.1e−2	**4.3**	7.9	1.8e0	1.8e0	11.5	27.5	1.2e2	1.3e2
40	16757.4	20146.0	2.4e4	1.5e5	6.3	20.8	1.8e−2	3.1e−2	**2.4**	7.6	2.5e0	2.6e0	5.5	15.9	2.5e2	2.7e2
45	19621.4	24528.0	6.8e4	3.8e5	4.8	12.3	1.6e−2	1.9e−2	**2.8**	11.9	3.5e0	3.7e0	5.3	13.1	4.2e2	4.5e2
50	21338.0	25044.0	7.7e4	4.8e5	6.2	12.4	1.9e−2	3.0e−2	**3.8**	11.1	4.7e0	4.8e0	4.2	12.0	6.6e2	7.4e2
55	23019.6	25376.0	1.4e5	4.7e5	5.0	11.4	2.2e−2	3.9e−2	5.0	16.2	6.2e0	6.5e0	**3.5**	14.7	1.1e3	1.2e3
60	26079.5	31320.0	1.9e5	6.0e5	5.0	13.5	2.8e−2	4.4e−2	4.4	15.4	8.0e0	8.2e0	**3.9**	20.8	1.7e3	1.8e3

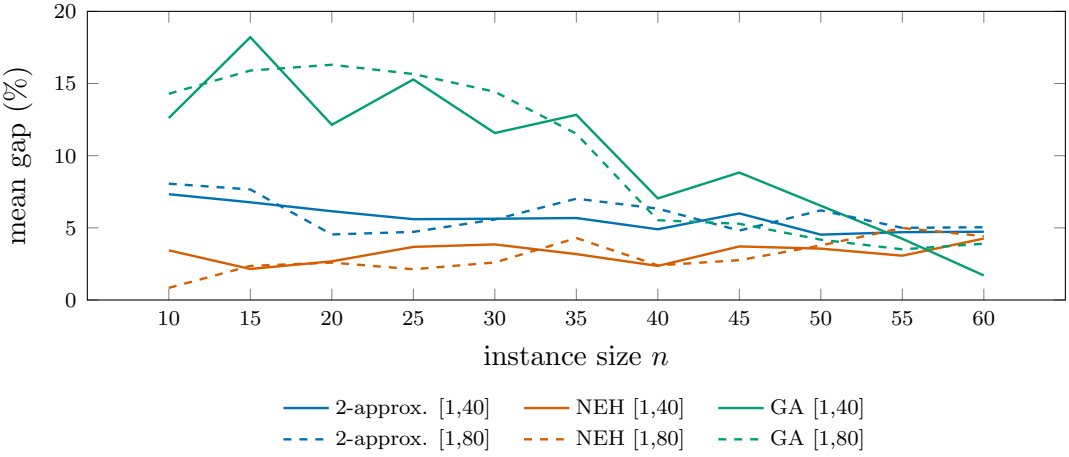


Figure 6: Mean relative gap of the 2-approximation and the two heuristics on the small scale, against the optimum the model returned. A solid line is $a_j, b_j \in [1, 40]$, a dashed line $a_j, b_j \in [1, 80]$.

6.3 The large scale

The three heuristics exhibit increasingly distinct performance as the instance size grows. The 2-approximation is the better constructive method, achieving a mean gap of 2.67% across the 32 configurations, a maximum configuration-level mean gap of 4.42%, and a worst-instance gap of 12.92%. In contrast, NEH has a higher mean gap of 5.06%, with relatively stable performance across instance sizes and a worst-instance gap of 10.77%. The GA shows a markedly different trend, with its mean gap decreasing from 18.33% to 0.62% under [1,40] and from 16.01% to 0.82% under [1,80]. It outperforms NEH at 220 jobs under both ranges and surpasses the 2-approximation at 220 jobs under [1,40] and 240 jobs under [1,80]. Across the 640 instances, the GA achieves the best objective value in 328 instances, compared with 250 for the 2-approximation and 62 for NEH. These improvements come at a substantially higher computational cost: the GA requires an average of 42.2 s per instance, with a maximum of 129.6 s, compared with 0.74 s for NEH and 0.09 ms for the 2-approximation. Its largest gaps occur among smaller instances, reaching 18.33% as a configuration-level mean at 100 jobs and 38.46% on an individual 120-job instance.

The two processing-time ranges yield the same overall ranking of the three heuristics in terms of mean gap. The 2-approximation remains the best-performing method, with mean gaps of 2.74% and 2.59% under [1,40] and [1,80], respectively, compared with 4.75% and 5.36% for NEH. The GA achieves intermediate mean gaps of 4.44% and 4.96%, outperforming NEH under both ranges but remaining behind the 2-approximation in overall average performance. Its advantage over the 2-approximation emerges only at the largest instance sizes. The detailed results for both processing-time ranges are reported in Tables 4 to 5, while Figure 7 illustrates the evolution of the gaps across instance sizes.

Overall, the large-scale results show that the constructive heuristics remain relatively close to the lower bound, whereas the GA progressively improves and eventually surpasses both. The methods are therefore better viewed as complementary rather than as competitors in a single ranking. The 2-approximation is by far the most computationally efficient method, running approximately six orders of magnitude faster than the GA on large instances. Its small empirical gaps suggest that the theoretical guarantee in Theorem 4.2 is conservative for the tested instances. NEH offers a different balance: its initial sequence is constructed using processing times alone, whereas its insertion decisions account for job weights. This explains its competitive performance on smaller instances, although its gap remains relatively stable as the instance size increases.

The dynamic programming algorithm and the GA address different computational challenges beyond those handled by the constructive heuristics. The dynamic program provides optimality certificates for instances with two weight classes, solving instances with up to 100 jobs under [1,40] in the tested settings, with an average running time of 1.47 s for 40 jobs. Its applicability is nevertheless limited by the number of weight classes and instance size. The GA, in contrast, does not provide optimality certificates and requires more than 50 times the average running time of NEH. Nevertheless, it achieves the best objective value in 328 of the 640 large-scale instances and handles instances with up to 30 distinct weight values. These results highlight the different roles of the two methods: the dynamic program certifies optimality on restricted instances, whereas the GA seeks high-quality solutions for larger and more general instances. One possible explanation for the GA's slower improvement is its initialization strategy. Since Johnson's rule ignores job weights, early generations may need to recover the weighted structure through evolutionary search.

Two qualifications are important when interpreting these results. First, the large-scale gaps are measured relative to the lower bound LB defined in (31), rather than the unknown optimum. Since LB is tight only when the heaviest jobs also appear earliest in the sequence, these gaps provide a consistent basis for comparing the methods but do not directly establish their distances from optimality. Second, the relative performance of the GA depends on the processing-time range and instance size. Although its mean gap is higher than that of the 2-approximation under both ranges, it eventually surpasses the latter on the largest instances.

A promising direction for improving the GA is to seed its initial population with the schedule generated by the 2-approximation. This would provide a weight-aware initial solution at negligible additional computational cost and could potentially reduce the instance size at which the GA begins to outperform the constructive heuristics.

Table 4: The large scale, $a_j, b_j \in [1, 40]$: 2-approx., NEH, GA

n	2-approx.				NEH				GA			
	gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
100	**4.4**	10.0	3.4e−2	6.2e−2	5.2	9.7	3.7e1	6.3e1	18.3	29.2	9.7e2	1.1e3
120	**3.5**	6.5	4.8e−2	1.1e−1	4.8	10.0	6.1e1	6.3e1	13.8	32.1	1.8e3	1.8e3
140	**3.0**	6.6	5.8e−2	1.2e−1	5.3	8.5	9.7e1	1.0e2	9.3	20.6	3.1e3	3.2e3
160	**3.4**	7.5	6.7e−2	1.3e−1	4.2	7.4	1.4e2	1.5e2	6.5	14.4	5.1e3	5.3e3
180	**2.6**	7.8	7.1e−2	9.8e−2	4.9	7.6	2.0e2	2.2e2	5.3	13.8	8.0e3	8.4e3
200	**2.6**	5.6	7.5e−2	1.2e−1	4.4	7.6	2.7e2	2.9e2	4.5	15.7	1.2e4	1.3e4
220	3.4	8.6	8.7e−2	1.1e−1	5.0	8.2	3.6e2	3.8e2	**3.0**	11.7	1.8e4	1.8e4
240	**2.8**	5.9	8.5e−2	1.1e−1	4.7	7.3	4.9e2	5.1e2	2.8	7.4	2.6e4	2.7e4
260	2.6	6.0	1.0e−1	1.5e−1	5.4	8.7	5.7e2	5.8e2	**1.6**	4.7	3.2e4	3.3e4
280	2.5	7.8	9.7e−2	1.2e−1	5.1	7.7	7.2e2	7.4e2	**0.8**	3.9	4.3e4	4.4e4
300	2.0	4.9	1.1e−1	1.8e−1	4.9	8.6	9.0e2	9.1e2	**0.9**	5.0	5.3e4	5.4e4
320	3.0	7.6	1.1e−1	1.2e−1	4.4	7.4	1.1e3	1.1e3	**1.2**	6.3	6.5e4	6.6e4
340	1.5	3.7	1.3e−1	2.4e−1	4.7	7.1	1.3e3	1.3e3	**1.2**	8.0	7.8e4	7.9e4
360	2.2	5.5	1.3e−1	1.6e−1	4.7	7.0	1.6e3	1.6e3	**0.9**	7.1	9.3e4	9.6e4
380	2.3	6.2	1.4e−1	1.5e−1	4.5	8.7	1.9e3	1.9e3	**0.3**	1.5	1.1e5	1.1e5
400	2.0	5.9	1.5e−1	1.7e−1	3.9	8.0	2.2e3	2.2e3	**0.6**	2.7	1.3e5	1.3e5

Table 5: The large scale, $a_j, b_j \in [1, 80]$: 2-approx., NEH, GA

n	2-approx.				NEH				GA			
	gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
100	**3.6**	8.9	4.1e−2	1.3e−1	4.6	10.8	3.6e1	4.1e1	16.0	30.1	9.6e2	1.0e3
120	**4.4**	12.9	4.3e−2	7.1e−2	5.4	9.3	6.1e1	6.2e1	17.1	38.5	1.8e3	1.8e3
140	**3.5**	10.7	5.2e−2	9.4e−2	4.4	7.7	9.6e1	9.9e1	12.6	25.3	3.1e3	3.2e3
160	**3.2**	11.4	6.8e−2	1.1e−1	5.8	9.9	1.4e2	1.5e2	8.1	25.8	5.1e3	5.2e3
180	**3.0**	7.3	6.7e−2	9.6e−2	5.3	9.0	2.1e2	2.5e2	5.9	14.3	8.3e3	8.5e3
200	**2.7**	8.6	9.3e−2	2.5e−1	4.7	7.2	2.8e2	2.9e2	4.9	10.1	1.2e4	1.3e4
220	**2.9**	8.0	8.0e−2	1.1e−1	4.8	8.7	3.8e2	4.0e2	3.6	9.0	1.8e4	1.9e4
240	2.9	6.4	8.9e−2	1.2e−1	6.0	8.9	4.5e2	4.8e2	**1.9**	6.5	2.4e4	2.5e4
260	1.8	5.3	9.1e−2	1.1e−1	5.2	9.3	5.7e2	5.8e2	**1.6**	8.0	3.2e4	3.3e4
280	2.3	4.8	1.0e−1	1.3e−1	5.5	9.2	7.2e2	7.3e2	**0.8**	4.3	4.3e4	4.4e4
300	2.8	6.3	1.0e−1	1.2e−1	5.4	7.5	9.0e2	9.1e2	**2.0**	11.8	5.3e4	5.4e4
320	1.9	4.7	1.2e−1	2.3e−1	5.7	8.5	1.1e3	1.1e3	**1.4**	6.1	6.5e4	6.6e4
340	1.4	3.0	1.2e−1	1.3e−1	5.6	8.6	1.3e3	1.4e3	**0.8**	3.3	7.8e4	8.0e4
360	1.7	3.2	1.3e−1	1.5e−1	5.8	8.2	1.6e3	1.6e3	**1.1**	7.3	9.3e4	9.4e4
380	1.8	4.0	1.4e−1	1.6e−1	5.9	9.1	1.9e3	1.9e3	**0.8**	2.9	1.1e5	1.1e5
400	1.7	4.6	1.5e−1	2.1e−1	5.8	9.1	2.2e3	2.3e3	**0.8**	2.9	1.3e5	1.3e5

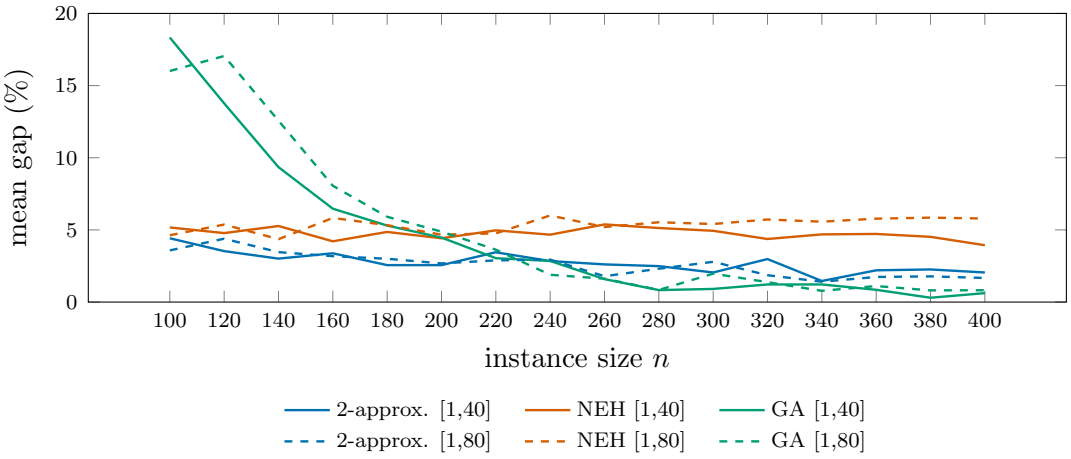


Figure 7: Mean relative gap of the 2-approximation and the two heuristics on the large scale, against the lower bound LB. A solid line is $a_j, b_j \in [1, 40]$, a dashed line $a_j, b_j \in [1, 80]$.

7 Conclusion

This paper studies the two-machine flow shop scheduling problem with the maximum weighted completion time as its objective. The problem is strongly NP-hard, and an optimal schedule exhibits a block structure: jobs are partitioned into one block for each distinct weight, Johnson's order is followed within each block, and the blocks are arranged in non-increasing order of weight. This structural property leads to a dynamic programming algorithm that eliminates dominated states and runs in $O(nK2^K V^{3K-3})$ time, a 2-approximation algorithm with an $O(n \log n)$ running time, a fully polynomial-time approximation scheme (FPTAS) for a fixed number of weight classes, and two heuristics—an adapted NEH algorithm and a genetic algorithm—to handle instances beyond the practical reach of the exact methods.

The computational study illustrates the complementary strengths of these methods. The MILP model solves 436 of the 440 small-scale instances to optimality, while the dynamic programming algorithm certifies optimality for instances with $K = 2$ and up to 100 jobs under [1,40] and 80 jobs under [1,80], within the 600-s time limit. On small instances, the 2-approximation and NEH achieve average gaps of 5.77% and 3.14%, respectively, relative to the reference values. On large instances, their average gaps relative to the lower bound are 2.67% and 5.06%, respectively. Although the genetic algorithm performs less favorably on small instances, it eventually outperforms both constructive heuristics at approximately 220–240 jobs. These results suggest that the methods are complementary rather than competing alternatives: the constructive heuristics offer computational efficiency, the exact methods provide optimality certificates within their applicable ranges, and the genetic algorithm can deliver further solution improvements on larger instances.

Three directions for future research emerge from this study. First, the problem can be extended to more than two machines. The existing MILP formulation provides a starting point for this extension, although the single-permutation property established for the two-machine case does not generally hold for $m > 2$. The extended model must therefore explicitly account for machine-specific sequencing decisions and precedence constraints between operations. Second, for problems with more weight classes, a branch-and-bound framework offers a promising alternative to the dynamic programming algorithm. The block structure can guide branching decisions, while the lower bounds LB_1, LB_2, and LB_3 developed in Section 4.3 can be used to evaluate partial schedules and prune unpromising nodes. Third, the proposed heuristics can be extended to distributed flow shops, where job assignment to shops and job sequencing must be determined jointly. Since both heuristics rely on processing times and job weights, NEH can be adapted to evaluate insertions across candidate shops, while the genetic algorithm can encode shop assignments together with job sequences.

References

Becker, T., Neufeld, J. S., and Buscher, U. (2025). The distributed flow shop scheduling problem with inter-factory transportation. *European Journal of Operational Research*, 322(1):39–55.

Chai, X., Lu, L., Li, W., and Zhang, L. (2018). Best-possible online algorithms for single machine scheduling to minimize the maximum weighted completion time. *Asia-Pacific Journal of Operational Research*, 35(6):1850048.

Della Croce, F. and Schau, Q. (2026). Revisiting Johnson's rule for minimizing makespan in the two-machine flow shop scheduling problem. *Operations Research Letters*, 69:107514.

Fan, B., Lu, L., and Shi, Z. (2026). On-line single-machine scheduling to minimize the weighted makespan under (dis)agreeability conditions. *Journal of Industrial and Management Optimization*, 22(8):4138–4150.

Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of $\mathcal{NP}$-Completeness*. W. H. Freeman and Company, San Francisco, CA.

Hall, L. A. (1994). A polynomial approximation scheme for a constrained flow-shop scheduling problem. *Mathematics of Operations Research*, 19(1):68–85.

Hnaien, F. and Arbaoui, T. (2023). Minimizing the makespan for the two-machine flow shop scheduling problem with random breakdown. *Annals of Operations Research*, 328(2):1437–1460.

Johnson, S. M. (1954). Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68.

Li, W. (2015). A best possible online algorithm for the parallel-machine scheduling to minimize the maximum weighted completion time. *Asia-Pacific Journal of Operational Research*, 32(4):1550030.

Liu, L. and Urgo, M. (2024a). A branch-and-bound approach to minimise the value-at-risk of the makespan in a stochastic two-machine flow shop. *International Journal of Production Research*, 62(6):2107–2123.

Liu, L. and Urgo, M. (2024b). Robust scheduling in a two-machine re-entrant flow shop to minimise the value-at-risk of the makespan: branch-and-bound and heuristic algorithms based on Markovian activity networks and phase-type distributions. *Annals of Operations Research*, 338(1):741–764.

Lu, L., Zuo, L., Zhang, L., and Ou, J. (2025). Order acceptance and scheduling with weighted makespan. *4OR*, 23(3):247–267.

Mosheiov, G. and Sarig, A. (2023). A note on lot scheduling on a single machine to minimize maximum weighted tardiness. *Journal of Combinatorial Optimization*, 45(5):128.

Nong, Q. Q., Cheng, T. C. E., and Ng, C. T. (2011). Two-agent scheduling to minimize the total cost. *European Journal of Operational Research*, 215(1):39–44.

Pinedo, M. L. (2022). *Scheduling: Theory, Algorithms, and Systems*. Springer, Cham, Switzerland, 6 edition.

Puka, R., Skalna, I., Duda, J., and Stawowy, A. (2024). Deterministic constructive vN-NEH+ algorithm to solve permutation flow shop scheduling problem with makespan criterion. *Computers & Operations Research*, 162:106473.

Qiao, Y., Wu, N., He, Y., Li, Z., and Chen, T. (2022). Adaptive genetic algorithm for two-stage hybrid flow-shop scheduling with sequence-independent setup time and no-interruption requirement. *Expert Systems with Applications*, 208:118068.

Shabtay, D. and Gerstl, E. (2024). Coordinating scheduling and rejection decisions in a two-machine flow shop scheduling problem. *European Journal of Operational Research*, 316(3):887–898.

Shao, W., Shao, Z., and Pi, D. (2021). Effective constructive heuristics for distributed no-wait flexible flow shop scheduling problem. *Computers & Operations Research*, 136:105482.

Sun, R. (2024). Randomized approximation schemes for minimizing the weighted makespan on identical parallel machines. *Journal of Combinatorial Optimization*, 47(3):34.

Wang, J., Wang, G. G., Tian, F. M., Gong, D. W., and Pedrycz, W. (2023). Solving energy-efficient fuzzy hybrid flow-shop scheduling problem at a variable machine speed using an extended NSGA-II. *Engineering Applications of Artificial Intelligence*, 121:105977.

Yuan, J., Ng, C. T., and Cheng, T. C. E. (2020). Scheduling with release dates and preemption to minimize multiple max-form objective functions. *European Journal of Operational Research*, 280(3):860–875.

Zhang, H., Lu, L., and Yuan, J. (2025). Online scheduling on an unbounded parallel-batch machine to minimize the weighted makespan. *Journal of Combinatorial Optimization*, 49(1):6.

Zheng, F., Li, N., Liu, M., and Xu, Y. (2025). Single machine lot scheduling to minimize maximum weighted completion time. *Journal of Combinatorial Optimization*, 50(1):1.

Zuo, L., Lu, L., and Zhang, L. (2026a). Single machine scheduling with a restricted weight-modifying activity to minimize the weighted makespan. *International Journal of Foundations of Computer Science*, 37(5):693–707.

Zuo, L., Zhang, J., Shi, Z., and Fan, B. (2026b). Single machine scheduling with a restricted rate-modifying activity to minimize the weighted makespan. *Discrete Applied Mathematics*, 380:89–100.